



Wissenschaftliches Rechnen mit Python

Skriptum

Copyright (C) 1998 - 2024, FH Vorarlberg GmbH

Dieses Dokument ist urheberrechtlich geschützt.

Alle Rechte, auch die der Übersetzung, des Nachdrucks und Vervielfältigung durch Kopieren oder Scannen sowie der Speicherung in Retrieval-Systemen des gesamten Dokumentes oder Teilen daraus, sind FH Vorarlberg GmbH vorbehalten.

Kein Teil des Dokumentes darf ohne schriftliche Genehmigung von FH Vorarlberg GmbH in irgendeiner Form (Fotokopie, Mikrofilm oder ein anderes Verfahren), auch nicht für Zwecke der Unterrichtsgestaltung, reproduziert oder unter Verwendung elektronischer Systeme gespeichert, verarbeitet, vervielfältigt oder verbreitet werden.

Die Weitergabe an Dritte ist nur mit ausdrücklicher Erlaubnis von FH Vorarlberg GmbH gestattet.

Alle Marken und Produktnamen sind Warenzeichen oder eingetragene Warenzeichen der jeweiligen Titelhälter.

Versionsgeschichte

Datum	Version	Änderung
2024-11-12	0.3.0	• Aktualisierung der Roadmap • Erweiterung um Dreiphasensystem animiert.
2024-11-05	0.2.0	• Ergänzung des Glossars. • Ergänzung um weitere Aufgaben.
2024-10-23	0.1.0	• Kap. <i>Librarys fürs wissenschaftliche Rechnen</i>. • Umstrukturierungen. • Korrekturen im Layout. • Aktualisierung der Roadmap.
2024-10-22	0.0.0	Erstausgabe.

Inhaltsverzeichnis

1	Glossar.....	1
2	Einleitung.....	3
2.1	Willkommen in der Lehrveranstaltung Wissenschaftliches Rechnen mit Python!.....	3
2.2	Organisatorisches.....	3
2.2.1	Eckdaten.....	3
2.2.2	Lehrbeauftragter.....	3
2.2.3	Arbeitsmittel.....	3
2.2.3.1	Ihr Computer.....	3
2.2.3.2	Unser Server.....	4
2.3	Ziele dieser Lehrveranstaltung (LV).....	4
2.3.1	Lernergebnisse.....	4
2.3.2	Lehrinhalte.....	4
2.3.3	Didaktik.....	4
2.4	Nicht-Ziele dieser LV.....	5
2.5	Prüfung.....	5
2.6	Anerkennung.....	5
2.7	Anwesenheit.....	5
2.8	Evaluation.....	5
2.9	Empfohlene Fachliteratur und andere Ressourcen.....	5
2.9.1	Bücher.....	5
2.9.2	Web Ressourcen zu Python.....	6
2.9.3	Ressourcen zu Python an der FH Vorarlberg.....	7
2.10	Dein Lernerfolg.....	7
2.10.1	Mitlernen.....	7
2.10.2	Selbstlernanteil.....	7
2.10.2.1	Aktive Teilnahme an der Lehrveranstaltung.....	8
2.10.2.2	Eigenverantwortung.....	8
2.10.2.3	Kommunikation.....	8
2.11	Roadmap 2024-WS-25.....	9
3	Programmieren, was ist das?.....	11
4	Tools.....	13
4.1	IDEs.....	13
4.1.1	Anaconda-Distribution.....	13
4.1.2	Idle.....	13
4.1.3	Jupyter oder Jupyter Lab.....	13
4.1.4	Mu.....	14
4.1.5	NetBeans.....	14
4.1.6	PyCharm.....	14
4.1.7	PyDev.....	14

4.1.8	Spyder.....	14
4.1.9	Thonny.....	14
4.1.10	Visual Studio Code (VS Code).....	14
4.1.11	Wing.....	14
4.2	Jupyter.....	14
4.2.1	Lokale Installation.....	14
4.2.2	Remote „Installation“.....	14
4.2.2.1	Einführung, Tipps und Tricks.....	15
5	Einführung in Python – Erste Schritte.....	17
5.1	Warum Python?.....	17
5.2	Erste Schritte in Python.....	18
5.2.1	Hello, World!.....	18
5.2.2	py- und pyc-Files.....	18
5.2.3	Sachen zum Lachen.....	18
5.2.4	Einfach zum Nachdenken - The Zen of Python, by Tim Peters.....	19
5.2.5	Statisch typisierte vs. dynamisch typisierte Programmiersprachen.....	21
5.2.6	Building Blocks.....	21
5.2.6.1	Variable.....	21
5.2.6.2	Datentypen und -strukturen.....	22
5.2.6.2.1	int - Ganzzahlen.....	22
5.2.6.2.2	float - Fließkommazahlen.....	22
5.2.6.2.3	String.....	23
5.2.6.2.4	Tuples.....	24
5.2.6.2.5	list - Listen.....	24
5.2.6.2.6	Iterativer Zugriff auf Elemente einer Liste.....	25
5.2.6.2.7	List Comprehensions.....	25
5.2.6.2.8	dict - Dictionaries.....	26
5.2.6.2.9	Sets (Mengen).....	26
5.2.6.2.10	Files	27
5.2.6.2.11	Klassen	27
5.2.6.3	Ausdrücke und Anweisungen.....	27
5.2.6.4	Steuerung des Programmflusses.....	27
5.2.6.5	Programmierstil.....	28
5.2.7	Idioms.....	29
5.2.7.1	Werte tauschen.....	29
5.2.7.2	Aus einer oder mehreren Listen mach' eine Liste.....	30
5.2.7.3	Aus zwei Listen mach' (durch Verschränken) eine Liste von Paaren.....	30
5.2.7.4	Aus einer Liste von Paaren mach' zwei Listen.....	30
5.2.7.5	Zugriff auf Listenelemente mittels negativer Indices.....	30
5.2.7.6	Daten aus/in Objekte/n lesen/schreiben.....	30

6	Grundlagen.....	33
6.1	Variable und Datentypen bzw. -strukturen.....	33
6.1.1	Variable.....	33
6.1.2	Numerische Datentypen.....	34
6.1.2.1	int – Ganzzahlen.....	34
6.1.2.2	float – Fließkommazahlen.....	35
6.1.2.3	complex - Komplexe Zahlen.....	35
6.1.3	Sequentielle Datentypen.....	36
6.1.3.1	str - Strings (Zeichenketten).....	36
6.1.3.2	list – Listen.....	37
6.1.3.2.1	Iterativer Zugriff auf Elemente einer Liste.....	38
6.1.3.2.2	List Comprehensions.....	38
6.1.3.3	tuple – Tuples.....	39
6.1.3.4	Rechnen mit Sequenztypen?.....	39
6.1.3.4.1	Strings.....	39
6.1.3.4.2	Listen.....	40
6.1.3.4.3	Tuples.....	40
6.1.3.4.4	Rechnen mit Listen und Tuples.....	40
6.1.4	Indexing, Slicing und weitere Operationen auf sequentielle Datentypen.....	42
6.1.5	Assoziative Datentypen.....	43
6.1.5.1	dict – Dictionaries.....	43
6.1.5.2	set – Mengen.....	44
6.1.6	Mutable vs. immutable.....	44
6.1.7	Übungen zu Variablen und Datentypen.....	45
6.1.7.1	Wie oft wird die Funktion print ausgeführt?.....	45
6.1.7.2	Wie lautet der String, der schlussendlich ausgegeben wird?.....	45
6.1.7.3	Schreiben Sie ein Skript, das ausgibt, welchen Typs die Variable A, B und C sind!.....	45
6.1.7.4	Was gibt die folgende Programmsequenz aus?.....	45
6.1.7.5	Und was gibt die folgende Programmsequenz aus?.....	45
6.1.7.6	Schreiben Sie ein Programm, das alle Ganzzahlen zwischen 1 und 10 inklusive aufaddiert und das Ergebnis ausgibt!.....	45
6.1.7.7	Warum funktioniert folgende Programmsequenz nicht?.....	45
6.1.7.8	Schreiben Sie ein Programm, das alle geraden Zahlen der Liste und auf jeden Fall die beiden letzten Zahlen ausgibt!.....	46
6.1.7.9	Schaltjahrbestimmung: Schreiben Sie ein Programm, das vom User eine Jahreszahl entgegennimmt und dann über eine Funktion auf dem Bildschirm ausgibt, ob es sich um ein Schaltjahr handelt oder nicht!.....	46
6.1.7.10	Harmonische Reihe: Schreiben Sie eine Funktion, die die Harmonische Reihe berechnet!.....	46
6.2	Programmstruktur.....	46
6.2.1	Lineare Programmstruktur.....	46
6.2.2	Funktionen.....	47

6.2.2.1	Motivation und Definition.....	47
6.2.2.2	Formal- und Aktualparameter.....	47
6.2.2.3	Return-Werte.....	48
6.2.3	Verzweigungen mit if.....	48
6.2.4	Wiederholungsstrukturen.....	49
6.2.4.1	Schleifen mit while.....	49
6.2.4.2	Schleifen mit for.....	49
6.2.5	Callbacks.....	50
6.2.6	Übungen zu Programmstruktur.....	50
6.2.6.1	Temperaturrechner.....	50
6.3	Exceptions.....	50
6.4	Files.....	51
6.4.1	Übungen zu Files.....	52
6.4.1.1	File-Lister.....	52
7	Grundlagen der objektorientierten Programmierung (OOP).....	55
7.1	Einführung.....	55
7.2	Ein einführendes Beispiel.....	55
7.3	Kriterien objektorientierter Sprachen.....	56
7.4	Klassen.....	56
8	Grafische Ausgabe mit matplotlib.....	59
8.1	Dokumentation.....	59
8.2	Rezepte.....	59
8.2.1	Balkendiagramme plotten.....	59
9	Librarys fürs wissenschaftliche Rechnen.....	61
9.1	NumPy.....	61
9.1.1	Ein Beispiel zur Motivation.....	61
9.1.1.1	Grafisches Ausgabe der Funktion $y = \sin^2(x)$	61
9.1.1.1.1	Variante 1.....	61
9.1.1.1.2	Variante 1 + 2.....	61
9.1.2	Aufgabe sinc-Funktion.....	62
9.1.3	Matrizen – erste Schritte.....	62
9.2	SymPy.....	63
10	Python als Taschenrechner.....	65
10.1	Mengen 1.....	65
10.2	Mengen 2.....	65
10.3	Funktionen.....	65
10.3.1	Geradengleichung – Variante einfache Funktion.....	65
10.3.2	Geradengleichung – Variante objektorientiert.....	65
10.3.3	Quadratische Gleichung.....	65
10.3.4	Quadratische Gleichung mit cmath.....	66
10.3.5	Dreiecksfläche.....	66

10.3.6	Volumina.....	66
10.3.7	Maximum einer Parabel.....	66
10.4	Gleichungen und Gleichungssysteme.....	66
10.5	Vektorrechnung.....	67
10.5.1	Eigenwerte und Vektoren.....	67
10.6	Dreiphasensystem animiert.....	70
11	Literatur.....	71
12	Anhang: Fundstücke.....	73
13	Anhang: Dokumentenverzeichnis.....	75

Tabellenverzeichnis

Tab. 2.1: Python – Der Grundkurs.....	6
Tab. 2.2: Physik mit Python - Simulationen, Visualisierungen und Animationen von Anfnag an...	6
Tab. 2.3: Der Python-Kurs für Ingenieure und Naturwissenschaftler.....	6
Tab. 2.4: Algorithmen In Python.....	6
Tab. 2.5: Roadmap 2024-WS-25.....	9

Abbildungsverzeichnis

Fig. 1: Quelle: AYARS: Computational Physics with Python. 2013.....	33
Fig. 2.....	34
Fig. 3.....	57
Fig. 4.....	70

1 Glossar

Compiler

Programm, das in einer Programmiersprache verfasste Texte (= Programme) in Objektdateien übersetzt, die der Linker dann zu ausführbare Datei zusammenbindet.

Debugger

Programm, mit dem man Programme auf Fehler hin untersuchen kann:

- Schrittweise Ausführung
- Inspektion von Daten
- Breakpoints

Editor

Programm, mit dem man Programme schreibt, sichert und dann durch Compiler und Linker bzw. Interpreter weiterverarbeitet.

IDE

Integrated Development Environment - Integriert Entwicklungsumgebung, stellt mindestens Editor und Debugger z.V.

Linker

Programm, das von einem Compiler erzeugte Objektdateien zu einem ausführbaren Programm zusammenbindet.

2 Einleitung

2.1 Willkommen in der Lehrveranstaltung *Wissenschaftliches Rechnen mit Python!*

2.2 Organisatorisches

Flex-Wahlfächer (FWF) sind Wahlfächer aus einem umfangreichen Wahlfachkatalog. Ihr Name leitet sich aus der flexiblen Organisation ab.

- Pro Semester sind 2 ECTS für vorgesehen. Diese 2 ECTS können auch in einem Folgesemester absolviert werden, wo das Semester dann eben mehr als die (bisher) erforderlichen 30 ECTS aufweist. Am Ende des Studiums müssen Sie 180 ECTS geleistet haben, aber die Verteilung der ECTS ist flexibler.
- Ihre Wahl ist nicht ans Semester gebunden, in dem sie zuerst angeboten werden¹, wenngleich dieses Semester mit Bedacht gewählt worden ist².
- Ihre Wahl erfolgt im November bzw. April fürs jeweils nächste Semester.

2.2.1 Eckdaten

- FWF für die Studiengänge
 - [Elektronik und Informationstechnologie Dual](#) und
 - [Mechatronik](#)der [FH Vorarlberg](#)
- Erstes Semester
- Umfang
 - 2 ECTS
 - 2 SWS ILV

2.2.2 Lehrbeauftragter

[Franz Geiger](#), +43 5572 792-5801, V1 04

2.2.3 Arbeitsmittel

2.2.3.1 Ihr Computer

Installieren Sie Python auf Ihrem Rechner. Windows- und MacOS-User finden den Installer unter <https://www.python.org/downloads/>. Achten Sie darauf, dass sie nur eine Python-Version auf

¹ Die "Jahreszeit" bleibt allerdings: Ein des 3. Semesters kann im 5. Semester besucht werden, nicht aber im 4. Semester. Es versteht sich von selbst, dass nicht angerechnet werden können.

² Da [Python](#) bereits ab dem 1. Semester v.a. in Mathematik eingesetzt wird, scheint es am sinnvollsten dieses bereits im 1. Semester und nicht erst im 3. oder 5. Semester zu wählen.

dem System haben oder immer wissen, welche wann aktiviert wird.

Um Programme zu schreiben und auszuführen, benötigen Sie im Prinzip nur einen ASCII-Editor. Produktiver sind Sie mit einer sogenannten IDE (s. Kap. 4.1), die Sie lokal installieren.

2.2.3.2 Unser Server

Wenn Sie sich für Jupyter-Lab als IDE entscheiden, können Sie diese IDE lokal installieren, müssen aber nicht, denn eine Server-Installation von Jupyter-Lab finden Sie auf <https://jupyter.labs.fhv.at/hub/>. Probieren Sie sie bitte aus.

2.3 Ziele dieser Lehrveranstaltung (LV)

Diese LV führt Sie an die Verwendung von Python als wissenschaftlichem Taschenrechner heran.

2.3.1 Lernergebnisse

Die Studierenden können nach Abschluss dieser Lehrveranstaltung

- einfache **Python-Programme schreiben** und dafür mit ausgewählten Entwicklungsumgebungen umgehen
- mit den wichtigsten Python-Packages für wissenschaftliches Rechnen einfache **Aufgabenstellungen aus Mathematik, Physik, Elektrotechnik und Mechatronik** bearbeiten, d.h. sie können
 - mit der grundlegenden Datenstruktur `array` in verschiedenen Szenarien einsetzen
 - die Lösbarkeit von Gleichungssystemen beurteilen
 - Gleichungssystemen lösen
 - Vektorisierung einsetzen um Mehrgrößenprobleme lauffzeiteffizient zu lösen
 - Ergebnisse mit `matplotlib` visualisieren
 - komplexe Zahlen in der komplexen Ebene darstellen
 - komplexe Zahlen als zeitabhängige Vektoren interpretieren und visualisieren

2.3.2 Lehrinhalte

- Entwicklungsumgebungen (IDE)
- Grundlagen der Programmierung in Python
- Einführung in die Standard-Library von Python
- Einführung in Python-Packages für wissenschaftliches Rechnen
- Anwendungsbeispiele aus Mathematik, Physik und Elektrotechnik

2.3.3 Didaktik

Diese ist eine ILV, also eine Vorlesung mit Übungselementen. **Programmieren ist wie Radfahren, das man nur durch Tun lernt.** Sie können sich in dieser LV zeigen lassen, was es alles

gibt, wann und wie man es prinzipiell anwendet, aber tun, **tun müssen Sie es selber**. Und dazu haben Sie Zeit mehr als genug! Nützen Sie unsere Empfehlungen und bringen Sie auch gerne weitere, die Ihnen besonders geholfen haben (mit einer Bemerkung, warum).

2.4 Nicht-Ziele dieser LV

- Kein tiefergehendes OOP, dafür haben wir ein eigenes FWF
- Kein Python auf [Raspberry Pi](#)
- Kein [Micropython](#) bzw. [CircuitPython](#)
- Kein Distributed Computing ([MQTT](#), ...)

2.5 Prüfung

- Bewertung der Ausarbeitung und Präsentation einer Beispielanwendung (20 %)
 - Eigene Ideen
 - Katalog als Backup
- Schriftliche Prüfung (80 %)

Für eine positive Gesamtnote müssen **in jedem Prüfungsteil mindestens 50%** der Punkte erzielt werden.

2.6 Anerkennung

Eine Anerkennung dieser LV ist - wenn sie gewählt wird - **nicht möglich**.

2.7 Anwesenheit

Es besteht keine Anwesenheitspflicht.

2.8 Evaluation

Diskussion der LV in der Gruppe.

2.9 Empfohlene Fachliteratur und andere Ressourcen

2.9.1 Bücher

Python – Der Grundkurs	
Autor	Kofler, Michael
Verlag	2022, Rheinwerk Computing
ISBN	978-3-8362-8515-5
Anmerkung	Titel ist in der FHV-Bibliothek vorhanden, aber auch online verfügbar:

Python – Der Grundkurs	
	https://ebookcentral.proquest.com/lib/vorarlberg/reader.action?docID=6807936

Tab. 2.1: Python – Der Grundkurs

Physik mit Python - Simulationen, Visualisierungen und Animationen von Anfängern	
Autor	Natt, Oliver
Verlag	2020, Springer Spektrum
ISBN	1978-3-662-61273-6
Anmerkung	

Tab. 2.2: Physik mit Python - Simulationen, Visualisierungen und Animationen von Anfängern

Der Python-Kurs für Ingenieure und Naturwissenschaftler	
Autor	Steinkamp, Veit
Verlag	2021, Rheinwerk Technik
ISBN	978-3-8362-7316-9
Anmerkung	

Tab. 2.3: Der Python-Kurs für Ingenieure und Naturwissenschaftler

Algorithmen In Python	
Autor	Kopec, David
Verlag	2020, Rheinwerk Computing
ISBN	978-3-8362-7747-1
Anmerkung	

Tab. 2.4: Algorithmen In Python

2.9.2 Web Ressourcen zu Python

Von welchen Websites haben Sie besonders profitiert?

Schreiben Sie mir, ich nehme die URL gerne in dieses Dokument auf!!

Sie wollen sich selber helfen und fragen sich, welche Web Ressourcen gerade für "fortgeschrittene Anfänger" sinnvoll sein können? Hier eine kleine Auswahl:

<https://pythontutor.com/>

Hier können Sie sich zeigen lassen, was Ihr Code mit Ihrem Computer macht bezüglich Objekten, die angelegt werden, Speicherverbrauch u. dgl.

Getting Started With Python

- <https://stackoverflow.blog/2021/07/14/getting-started-with-python/>
- file:///atlantis/p/python/AMAs_programmierbeispiele_per_2021-09/SammelmappeHP.pdf

(lokal)

Fundgruben für weitere Beispiele

- <https://hpcodewarsbcn.com/>
- file:///atlantis/p/python/AMAs_programmierbeispiele_per_2021-09/SammelmappeJutge.pdf (lokal)
- [Learn to Code](#)

Statistik

- [Python Statistics Fundamentals: How to Describe Your Data](#)

Vermischtes

- [Learn to Code - Python Tutorial](#)
- [Python Website](#)
- [Python For Scientific Computing](#)
- [Scipy Lecture Notes](#)
- [Python for Engineers](#)
- [An Introduction to Python for Scientific Computing](#)
- [Python für Naturwissenschaftler](#)

2.9.3 Ressourcen zu Python an der FH Vorarlberg

Kollege Klaus Rheinberger unterhält eine schöne Begleit-Präsenz im Intranet der FHV zu seiner Lehrveranstaltung [Effiziente Netzwerke](#). Hier finden Sie eine Seite zu Python, voller Links, Tips, Code-Beispiele, wertvoller Packages - hier ist alles Python, nicht nur die eine Seite <https://energie.labs.fhv.at/~kr/effsys-en/Python.html> in <https://energie.labs.fhv.at/>.

Schauen Sie rein, es lohnt sich, unbedingt!

2.10 Dein Lernerfolg

2.10.1 Mitlernen

Die Inhalte sind aufbauend, der Lernerfolg stellt sich daher am ehesten ein, wenn du am Ball bleibst. Bitte arbeite auch vor jeder Lehrveranstaltung die geplanten Inhalte - sofern vorhanden - durch. Wiederhole gegebenenfalls die für diese Inhalte notwendigen Grundlagen!

2.10.2 Selbstlernanteil

Es ist bei diesen Lehrveranstaltungen nicht anders als bei allen anderen: Ein erheblicher Teil der Lehrveranstaltung ist Selbstlernanteil. Wie viel? Nun, die Rechnung ist

$$\text{Selbstlernanteil} = \text{ECTS} - \text{SWS}$$

Ein konkretes Beispiel – 4 ECTS, 2 SWS:

$$4 \cdot 25\text{ h} - 2 \cdot 15 \cdot 0.75\text{ h} \text{ ergibt } 66\text{ h Selbstlernanteil}$$

Du hast also nicht nur ausreichend Zeit, die Lehrveranstaltung vor- und nachzubereiten sowie auf die Prüfungen zu lernen, nein, das wird vorausgesetzt!

Oder 2 ECTS, 2 SWS wie in den meisten FWF:

$$2 \cdot 25 h - 2 \cdot 15 \cdot 0.75 h \text{ ergibt knapp } 28 h \text{ Selbstlernanteil}$$

2.10.2.1 Aktive Teilnahme an der Lehrveranstaltung

In Lehrveranstaltungen sind Sequenzen eingebaut, die deine aktive Teilnahme einfordern.

2.10.2.2 Eigenverantwortung

Wie bereits die Erklärung zum Selbstlernanteil durchblicken lässt: Wiederhole die Inhalte der Lehrveranstaltungen zeitnah und prüfe, ob du die Lernergebnisse erreicht hast. Es liegt in deiner Verantwortung, aktiv zu werden, falls Lücken bestehen. Dasselbe gilt für versäumte Lehrveranstaltungstermine: Es liegt in deiner Verantwortung, aufzuholen!

2.10.2.3 Kommunikation

Eine Reaktion auf eine asynchrone Anfrage (z.B. E-Mail) erfolgt in beide Richtungen innerhalb von 3 Werktagen (Ausnahmen: Urlaub, Krankheit u. dgl.). Als Reaktion gilt zumindest eine Empfangsbestätigung mit einer Ankündigung, wann mit einer inhaltlichen Reaktion gerechnet werden kann.

2.11 Roadmap 2024-WS-25

Termin	Datum	Inhalt
1	2024-10-15	• Einleitung • Programmieren, was ist das? Tools • Einführung in Python – Erste Schritte
2	2024-10-22	• Einführung in Python – Erste Schritte • Grundlagen • Hausübung: Quadratische Gleichung
		•
3	2024-11-05	• Diskussion Hausübung Quadratische Gleichung • Grundlagen • Grundlagen der objektorientierten Programmierung (OOP) • Programmieren: Temperaturrechner • <i>Make it Work</i> in der Übung • <i>Make it Right</i> als Hausübung
4	2024-11-11	• Diskussion Hausübung Temperaturrechner • Grafische Ausgabe mit matplotlib • Libraries fürs wissenschaftliche Rechnen • Hausübung Dreiphasensystem animiert
5	2024-11-19	
6	2024-11-26	
7	2024-12-03	
8	2024-12-10	
9	2024-12-17	
10	2025-01-14	Prüfung

Tab. 2.5: Roadmap 2024-WS-25

3 Programmieren, was ist das?

Ein Computer muss programmiert werden, er hat sonst keinen Nutzen.

Man kann sich das Programmieren als die Erstellung einer Reihe von **Handlungsanweisungen** vorstellen, die man z.B. einem Freund auch geben würde, wenn man möchte, dass er etwas ganz Bestimmtes auf eine ganz bestimmte Art tut.

Wollten wir, dass unser Freund einen Kuchen bäckt, könnten wir das so formulieren:

```
1 Wenn keine Zutaten da:  
2     Alle Zutaten einkaufen  
3  
4 Zutaten zu einem Teig verarbeiten  
5 Teig in eine Form geben  
6 Form mit Teig ins Backrohr schieben  
7 Backrohr einschalten auf 150 Grad  
8 Solange nicht 50 Minuten vergangen sind:  
9     Warten  
10  
11 Kuchen aus Backrohr holen  
12 Kuchen verzieren
```

Genauso wie man diese Handlungsanweisungen in der Sprache schreibt, die der Freund versteht, muss man den Computer programmieren in einer Sprache, die er versteht. Wir tun das in der Programmiersprache [Python](#). [Python](#) ist eine sehr vielseitige, aber leicht erlernbare Sprache. Es gibt noch [viele andere Programmiersprachen](#), z.B. *C++*, *Rust*, *Java* usw. usw.

Es ist bei der Einführung in eine neue Programmiersprache üblich, ein sogenanntes Hello-World-Programm zu schreiben. In [Python](#) würde das so aussehen:

```
13 print( "Hello World!")
```

Bei Ausführung dieses Programms wird - große Überraschung - am Bildschirm der Text `Hello world!` angezeigt. Die Handlungsanweisung - oder im Software-Jargon ausgedrückt das Statement - ist hier also `print` und heißt "drucke auf den Bildschirm".

Probier's einfach mal aus, starte auf der Konsole die Python-Konsole, gib diese Zeile ein und drück' dann ENTER.

Programme bestehen natürlich selten aus nur einer Zeile. Im Gegenteil, die Größe von Programmen wird gerne mit KLOCs angegeben und steht für [thousands of lines of code](#).

Und davon - nämlich wie wir zu größeren Programmen kommen - handelt der Rest dieses Skriptums.

4 Tools

4.1 IDEs

Bei den IDEs haben Sie eine relativ große Auswahl. Zwar reicht ein einfacher Text-Editor zum Programmieren in Python, aber mit einer IDE ist man doch produktiver, nicht zuletzt, weil die meisten einen Debugger mitbringen. Auf der anderen Seite gibt es kaum noch "einfache Text-editoren" und ausgestattet mit den entsprechenden Plug-ins bringen sie schon viel mit von dem, was einen ausmacht.

Eine Übersicht über verfügbare IDEs finden Sie auf <https://wiki.python.org/moin/IntegratedDevelopmentEnvironments>. Im Folgenden einige IDEs, die der Autor dieses Skriptums ausprobiert hat.

4.1.1 Anaconda-Distribution

Die [Anaconda Distribution](#) (ehem. *Anaconda Individual Edition*) ist keine IDE, sondern eine vollständige Python-Distribution, die bereits viele Pakete mitbringt, die man sonst nachinstallieren müsste, wenn man nur Python mit der Standard-Lib installierte. Die Auswahl orientiert sich an den Bedürfnissen von Wissenschaftlern. Kollegen aus der Energietechnik empfehlen die Installation dieses Programmpakets.

- Browser-basierend
- Unterstützt *Literate Programming*, also das „Verweben“ von Code und Text. Exportiert man in ein PDF, wird aus dem Programm ein Dokument!

Eine sehr gute Einführung in Jupyter-Lab finden Sie auf <https://energie.labs.fhv.at/~kr/effsys-en/Python.html>. Weitere Tipps und Tricks kann man u.a. unter [28 Jupyter Notebook Tips, Tricks, and Shortcuts](#) nachlesen.

4.1.2 Idle

Eingebaut in jede Python-Distro.

4.1.3 Jupyter oder Jupyter Lab

- Browser-basierend
- Unterstützt [Literate Programming](#), also das „Verweben“ von Code und Text. Exportiert man in ein PDF, wird aus dem Programm ein Dokument!

Eine sehr gute Einführung in Jupyter-Lab finden Sie auf <https://energie.labs.fhv.at/~kr/effsys-en/Python.html>. Weitere Tipps und Tricks kann man u.a. unter [28 Jupyter Notebook Tips, Tricks, and Shortcuts](#) nachlesen.

Seit 2023-SS haben wir eine eigene Server-Installation - <https://jupyter.labs.fhv.at> - bitte gleich ausprobieren!

Wie startet man Jupyter Lab in einem anderen Directory?

Geben Sie auf der Konsole

```
jupyter lab --notebook-dir
/YOUR_DIRECTORYNAME_HERE
ein.
```

Wo finde ich eine Liste der sog. Built-in Magic Commands?

Eine solche Liste finden Sie z.B. hier:

<https://ipython.readthedocs.io/en/stable/interactive/magics.html>.

4.1.4 Mu

Simpler Editor, mit dem man z.B. die [MicroBits](#) proggen kann; kein [Debugger](#).

4.1.5 NetBeans

Python-Plugin scheint schlecht gepflegt.

4.1.6 PyCharm

Alles Notwendige und (leider) noch mehr ist da.

4.1.7 PyDev

Bei [PyDev](#) handelt es sich um ein [Eclipse](#)-Plugin. Wer also mit Eclipse arbeitet, ist damit sicher gut bedient.

4.1.8 Spyder

Vollständige mit [Editor](#) und [Debugger](#). In [Anaconda](#) enthalten.

4.1.9 Thonny

In [Thonny](#) ist alles drin, alles dran: Vollständige Python-Distribution mit allen nötigen Tools. Nennt sich selbst *Python for beginners*, kann aber mittlerweile auch mit Plug-ins erweitert werden.

4.1.10 Visual Studio Code (VS Code)

[VS Code](#) kann wohl so ziemlich alle Programmiersprachen, die's gibt, durch Plugins ähnlich [Eclipse](#), aber noch "programmiersprachenagnostischer". Man könnte fast schon sagen, da gibt's nichts, das es nicht gibt.

Ein gute Einführung, wie man VS Code konfiguriert, um Python-Software zu entwickeln, findet sich hier: [Getting Started with Python in VS Code](#).

4.1.11 Wing

Schnell, v.a. der [Debugger](#). Alles Notwendige ist da, aber nicht mehr. Kostet.

4.2 Jupyter

4.2.1 Lokale Installation

Die lokale Installation hatten wir bereits im Kap. 4.1.3 beleuchtet.

4.2.2 Remote „Installation“

- Wir haben mit <https://jupyter.labs.fhv.at/> gleich 2 Vorteile:

- Wir können mit [Jupyter](#)-Lab arbeiten.
- Wir können auf einem Server des Hauses(!) arbeiten, ersparen uns also eine lokale³ Installation auf unseren Rechnern.

4.2.2.1 Einführung, Tipps und Tricks

Die Vorteile von [Jupyter](#) sind,

- das GUI ist ein Internet-Browser
- es unterstützt *Literate Programming*⁴, also das „Verweben“ von Code und Text. Exportiert man in ein PDF, wird aus dem Programm ein Dokument!

Eine sehr gute Einführung in Jupyter-Lab finden Sie auf <https://energie.labs.fhv.at/~kr/effsys-en/Python.html>. Weitere Tipps und Tricks kann man u.a. unter [28 Jupyter Notebook Tips, Tricks, and Shortcuts](#) nachlesen.

Soviel zur Bedienung. Aber auch fürs Schreiben der Programme gibt's einiges, das zu wissen uns das Leben erleichtert. Dazu gehören die [Magic Commands](#). Ein ganz wichtiges ist sicher **%matplotlib inline**. Am Beginn eines Notebooks werden Grafiken direkt im Worksheet platziert, ohne dass ein separater Dialog geöffnet wird
[\[https://ipython.readthedocs.io/en/latest/interactive/magics.html#magic-matplotlib\]](https://ipython.readthedocs.io/en/latest/interactive/magics.html#magic-matplotlib).

Weitere Tips auf [Make Your Notebooks Look Better](#), wo gezeigt wird, wie Sie Ihre Notebooks als wirklich ansprechende Dokumente darstellen.

3 Je nach Stand, auf dem das Betriebssystem samt installierter Apps ist, kann das immer mal mit Friktionen verbunden sein

4 "Literate programming is a programming methodology that combines a programming language with a documentation language, making programs more robust, more portable, and more easily maintained than programs written only in a high-level language. [\[https://web.stanford.edu/group/cslicpublications/cslicpublications/site/0937073806.shtml\]](https://web.stanford.edu/group/cslicpublications/cslicpublications/site/0937073806.shtml)"

5 Einführung in Python – Erste Schritte

5.1 Warum Python?

[Python](#) ist aus einer Schulungssprache, einer Sprache fürs Programmieren-Lernen [hervorgegangen](#). Sie zählt zu den Scripting-Sprachen, die wiederum dafür gedacht waren/sind, fertige Programmteile u/o Bibliotheken zu (größeren) Programmen zusammenzubauen. Prominente Vertreter sind hier [Perl](#) und ganz besonders [Tcl/Tk](#). Lesenswert diesebzüglich ist der Artikel von [Ousterhout Scripting: Higher Level Programming for the 21st Century](#).

[Python](#) hat sich wegen ihrer sehr guten Lesbarkeit (im Gegensatz zu Perl) und der mächtigen Sprachkonstrukte immer mehr zu deiner All-Purpose-Language entwickelt. In den letzten Jahren hat sie auch in Wissenschaft und Forschung Einzug gehalten. Dass sie [Open Source](#) ist, hat sicherlich nicht geschadet, allerdings nicht im Sinne, wie Sie vllt denken, also nicht im Sinne von free beer sondern im Sinne von free speech. Denn es gilt immer zu bedenken, wem eigentlich unsere Daten gehören - uns oder dem Hersteller der Software, mit denen wir sie bearbeiten...

Einen weiteren Beitrag zur steigenden Popularität hat aber auch das Motto [Batteries Included](#). Es ist ja nie nur die Programmiersprache, sondern auch immer "das Drumherum", die Bibliotheken, die [standardmäßig](#) oder von [Dritten](#) angeboten werden. Auch da brilliert Python.

Die Sprache [Python](#) hat ihren Namen nicht von der Schlange, sondern von der englischen Komikertruppe [Monty Python](#). [Guido van Rossum](#), der Schöpfer der Sprache [Python](#) hatte gerade eine erste Version des Kernels seiner neuen Programmiersprache fertig, als eine Monty-Python-Episode im TV lief, deren Fan er ist. Der Rest ist Geschichte, wie's so schön heißt.

[Python](#) ist eine dynamisch und stark typisierte Programmiersprache, die interpretiert wird (exakt: Der Source Code wird in Byte Code übersetzt, der dann vom Interpreter interpretiert wird). Reine Python-Programme laufen daher je nach Anwendung 10- bis 100-mal langsamer als solche, die in einer statisch typisierten / compilierten Sprache geschrieben sind, weil solche Programme mehr oder weniger direkt in Maschinencode übersetzt werden. Dafür ist die Zeit, die man für die Entwicklung von Python-Programmen braucht, wesentlich kürzer als jene, die man für die Entwicklung von Programmen in einer statisch typisierten Sprache benötigt.

Als Vorteile von [Python](#) können genannt werden:

- Einfache Syntax im besten Sinne des Wortes, ausdrucksstark
- Sehr gute Lesbarkeit der Programme (wie Pseudocode) - es sei denn, man verwendet das `typing`-Package :-)
- High-Level-Datentypen
- Interpretiert, daher kann auch interaktiv gearbeitet werden
- Keine Compiler-/Linker-Läufe notwendig
- Keine „einfachen“ Abstürze (Segmentation Faults) im Fehlerfall, sondern detaillierte Tracebacks
- [Open Source](#) und sehr große Community, sehr stark wachsend im akademischen Bereich

Nachteile:

- Keine Compiler-/Linker-Läufe notwendig
- Laufzeitbedarf; allerdings bietet [Python](#) auch Mechanismen, die hier Raum für Laufzeitverbesserungen bieten (s. z.B. [lru_cache](#)).

Auffällig:

- Keine Klammern - whitespace matters

```
def sqr( x):  
    return x*x
```
- self

Dazu kommen wir, wenn wir uns mit objektorientierter Programmierung beschäftigen (Kap. 7).

5.2 Erste Schritte in Python

5.2.1 Hello, World!

Legen Sie ein File `hello.py` an und schreiben Sie:

```
1 print( "Hello, World!")
```

Führen Sie das File auf der Konsole aus per

```
$ python hello.py
```

Die Ausgabe auf einem Linux-Rechner ist dann (unter Windows ist der Prompt ein anderer, sonst ist natürlich alles gleich)

```
$ python hello.py  
Hello, World!  
$
```

5.2.2 py- und pyc-Files

Python wird als interpretierte Sprache bezeichnet. Das stimmt so nicht ganz. Wenn wir die Standard-Implementierung *CPython* hernehmen, wird durch Übersetzen der Source-Files Byte-Code erzeugt (compiliert?), der dann von der [Virtual Machine](#) ausgeführt (interpretiert?) wird.

Die Source-Files besitzen die File-Endung `.py`. Die Files, die den Byte-Code enthalten, enden auf `.pyc`. Wie ein PYC-File analysiert werden kann, finden Sie z.B. auf [Stack Overflow](#) im Post [How can I understand a .pyc file content](#). Die Analyse ist 3-stufig, bis man den disassemblierten Code auf dem Bildschirm hat. Wenn Sie einfach nur sehen wollen, wie *irgendein* PYC-File aussieht, wie Python-Byte-Code prinzipiell aussieht, dann werden sie in der offiziellen Python-Dokumentation unter [dis - Disassembler for Python bytecode](#).

Wie eine Python-Implementierung vorgeht, um Source-Code auszuführen, ist ihr überlassen. Die Sprache macht hier keine Vorschriften. Zur Zeit gibt es eine C-Implementierung (Standard-Python, auch als *CPython* bezeichnet), eine Java-Implementierung ([Jython](#)), eine [.NET](#)-Implementierung ([IronPython](#)) und eine Python-Implementierung ([PyPy](#)), um nur die bekanntesten zu nennen. Mehr dazu finden Sie unter [Python Implementations](#).

5.2.3 Sachen zum Lachen

Geben Sie auf der Konsole mal Folgendes ein

```
>>> from __future__ import braces
```

Das Ergebnis wird sein:

```
File "<stdin>", line 1
SyntaxError: not a chance
>>>
```

5.2.4 Einfach zum Nachdenken - The Zen of Python, by Tim Peters

Wenn Sie in einer Python-Konsole den Text

```
2 import this
```

eingeben und ENTER drücken, erhalten Sie folgende Ausgabe (zur Historie siehe z.B. <https://mail.python.org/pipermail/python-list/1999-June/001951.html>)

Beautiful is better than ugly

Code, der nicht „schön“ ist, ist schwer zu warten und sogar fehleranfällig. Doch, was ist „schön“ in diesem Zusammenhang? Vllt. sollte man eher „einfach“ sagen. Gibt es Code-Teile, die man weglassen kann? Kann man diese Struktur des eines Programmabschnittes vereinfachen? Mit der Zeit entwickelt man eine Art Bauchgefühl dafür, dass etwas nicht stimmt, noch bevor man konkret sagen kann, was nicht stimmt.

Doch Vorsicht, wenn Sie es mit Code eines Anderen zu tun haben. Streichen Sie nicht voreilig Teile des Programms, weil Sie glauben, sie seien unnötig. Es kann gut sein, dass sich der Autor damals etwas dabei gedacht hat...

Explicit is better than implicit.

Keine versteckten Annahmen, keine Tricks. Drücken Sie aus, was Sie meinen, zeigen Sie, was der Code tut, machen Sie es offensichtlich!

„self“ oder der this-Pointer in C++ sind ebenfalls Beispiele hierfür.

Simple is better than complex.

Halten Sie's mit Einstein: Machen Sie die Dinge so einfach wie möglich (aber nicht einfacher).

Complex is better than complicated.

Wenn Sie komplexe Aufgabenstellungen zu lösen haben, kann Komplexes gleich auch mal kompliziert werden – es sei denn, man verwendet Code aus der Std.-Lib oder aus Paketen, die für bestimmte Anwendungsgebiete nicht mehr wegzudenken sind. NumPy ist so ein Paket:

```
1 N = 10000000
2 a = range( N)
3 b = range( N)
4 c = []
5 for i in range( N):
6     c.append( a[ i] + b[ i])
```

ist kompliziert gegenüber der Lösung mit NumPy:

```
7 import numpy
8 N = 10000000
9 a = numpy.arange( N)
10 b = numpy.arange( N)
11 c = a + b
```

Flat is better than nested.

Beginnt man seinen Code in Package-/Modul-Hierarchien zu organisieren, läuft man schnell Gefahr, in einer Art Bürokratie zu enden. Nehmen Sie als Beispiel eine Knowhow-Sammlung in einem Textdokument, in dem Sie Wichtiges hierarchisch organisieren. Schnell werden Sie

merken, je tiefer die Hierarchie wird, desto schwieriger ist, es Bestehendes zu finden und Neues einzuordnen.

Sparse is better than dense.

Packen Sie nicht zuviel Code in dafür verfügbaren Raum. Wieviel Programmcode passt in eine Zeile? Falsche Frage! Die Frage muss richtig lauten: Kann man soviel Programmcode (in einer) Zeile noch lesen im Sinne von Verstehen?

Readability counts.

Sie oder Ihre Kollegen werden Ihren Code wahrscheinlich öfters lesen als schreiben, glauben Sie nicht?

Python treibt das ins Extreme (wenigstens wird es von vielen so empfunden): Nicht Klammern sondern Einrückungen bestimmen den Programmfluss; keine schließenden Strichpunkte; Inline-Doku wird unterstützt.

Special cases aren't special enough to break the rules.

Wenn Sie Regularien folgen, versuchen Sie, das immer zu tun. Ausnahmen müssen wohlbegründet und gekennzeichnet sein.

Für Python bedeutet das z.B. alles ist ein Objekt, dynamische Typisierung, die meiste Funktionalität in externen Modulen wie der Standard-Library (Regular Expressions sind ein gutes Beispiel hierfür).

Although practicality beats purity.

Diesen Satz muss man im Zusammenhang mit dem letzten sehen. Am Schluss muss etwas funktionieren, zeitgerecht. Hier sind Sie gefordert, die richtige Balance zu finden.

Python bietet daher mehr als ein Programmiermodell: OOP, prozedural und funktional (im Sinne von Functional Programming, also seiteneffektfrei).

Errors should never pass silently.

Machen Sie sich früh Gedanken über ein anständiges Fehlerkonzept. Tun Sie bei Fehlern nicht einfach nichts, das heißt Sie früher als Sie denken.

Python unterstützt hier durch Exceptions und Tracebacks.

Unless explicitly silenced.

Auch dieser Satz steht wieder mit dem vorhergehenden im Zusammenhang. Wenn Sie nicht anders können, tun Sie's wenigstens explizit, also wohlbegründet und gekennzeichnet.

In the face of ambiguity, refuse the temptation to guess.

Dieser Satz ist vermutlich am besten übersetzt mit „Kein Herumprobieren!“ Sie müssen immer wissen, was Sie tun und warum.

Auch Python rät nicht. So können Strings und Integers nicht addiert werden ([Perl](#) „kann“ das, Allerdings kann *Python* Strings und Integers multiplizieren - und das ist in der Tat sinnvoll:

```
"#" * 3 == "###"
```

There should be one -- and preferably only one -- obvious way to do it.

Folgen Sie dem Principle Of Least Surprise, das Verhalten Ihres Programms (User-Ebene) oder Ihrer Software (Devel-Ebene) sollte offensichtlich sein. Vermeiden Sie also Redundanzen v.a. in APIs.

Although that way may not be obvious at first unless you're Dutch.

Python versucht, wenig zu überraschen, offensichtliche Lösungen anzubieten – jedenfalls aus Sicht des Schöpfers der Sprache und der ist Holländer...

Now is better than never.

Dieses Statement könnte man auch formulieren als „done is better than perfect“. Sollen die Dinge von Beginn an perfekt sein, nimmt man sie nie in Angriff!

Although never is often better than right now.

Allerdings muss Nachdenken schon erlaubt sein. Hierzu passen auch YAGNI – das Wikipedia mit „‘You aren't gonna need it’ (YAGNI) is a principle of extreme programming (XP) that states a programmer should not add functionality until deemed necessary.“ erklärt.

Bezüglich Python ist hiermit wohl gemeint, dass das Team um diese Programmiersprache sehr konservativ ist, wenn's um neue Sprachelemente geht (aber auch hier gibt's Ausnahmen). Es kommt auch nicht alles in die Standard-Library, das irgendwie praktisch erscheint.

If the implementation is hard to explain, it's a bad idea.

Hierzu muss nicht viel gesagt werden, außer...

If the implementation is easy to explain, it may be a good idea.

...dass es umgekehrt nicht unbedingt besser sein muss.

Namespaces are one honking great idea -- let's do more of those!

Python kennt die Namespaces built-in, global und local, unterstützt sie mit entsprechenden Sprachelementen und implementiert sie mit der grundlegenden Datenstruktur *Dictionary*.

5.2.5 Statisch typisierte vs. dynamisch typisierte Programmiersprachen

Statisch typisierte Programmiersprachen - zu ihnen gehören C, C++ (Achtung: C++ hat mit C so gut wie nichts zu tun, außer dass C++ unglücklicherweise die gleiche Syntax hat wie C!), C#, Go, Java, Kotlin, ...

Alle diese Sprachen haben Compiler und Linker, die Ihre Programme vorab auf Syntax- und Typ-Fehler überprüfen, bevor sie ein Binary daraus machen.

Dynamisch typisierte Sprachen können das gleiche erst zur Laufzeit leisten. Darüber hinaus arbeiten dynamisch typisierte Programmiersprachen nicht mit Typen, sondern mit "Protokollen". Wenn also zwei Objekte die gleichen Methoden haben, die gleich ausgeführt werden (gleiche Parameterliste), dann sind sie für dynamisch typisierte Programmiersprachen gleich. Die Python-Community hat den Spruch geprägt, "If it walks like a duck, if it quaks like a duck, it is a duck." Ist praktisch, aber nicht ungefährlich. Aber auch darauf gibt's eine Antwort: "Python, das ist Programmieren für Erwachsene." ...

5.2.6 Building Blocks

5.2.6.1 Variable

Eine Variable ist ein Speicherbereich einer bestimmten Länge, gemessen in Bytes (= 8 Bits), der Daten enthält. Speicherbereiche werden über Adressen angesprochen. In höheren Programmiersprachen braucht man nicht mit Speicheradressen zu hantieren, sondern kann diesen Speicherbereichen Namen geben. Solche Namen könnte man auch – und andere Programmiersprachen wie C und C++ tun das - als Pointer bezeichnen. Die Adressen der Speicherbereiche, auf die die Namen "zeigen", können wir uns mit der Funktion `id()` anzeigen lassen.

Höhere Programmiersprachen verbinden mit solchen Speicherbereichen einen Typ wie Integer oder Float. In statisch typisierten Programmiersprachen muss einer Variablen ein Typ fest zuge-

ordnet werden. In dynamisch typisierten Sprachen ist das nicht notwendig, weil Namen einfach nur eine Art Labels sind, die an Speicherbereiche gebunden werden können, eben auf sie zeigen.

Rechner raus und – machen!

Lassen Sie uns das gemeinsam „erforschen“!

```
>>> i = 42
>>> id( i)
139791546582544
>>> i = 43
>>> id( i)
139791546582576
>>> j = i
>>> id( i)
139791546582576
>>> id( j)
139791546582576
>>>
```

Erklärungen?

5.2.6.2 Datentypen und -strukturen

Neben den "gängigen" Datentypen wie Integer, Float, String bietet Python Listen, Dictionaries, Mengen und noch einige mehr.

5.2.6.2.1 int - Ganzzahlen

Ganzzahlen, hierzu gibt's nicht viel zu sagen, außer - und das erscheint mir nicht unwichtig - dass Ganzzahlen in Python nicht überlaufen können:

```
>>> 42**42
150130937545296572356771972164254457814047970568738777235893533016064
>>> type( _)
<class 'int'>
>>>
```

Wir können vom System Informationen über diesen Datentyp erhalten:

```
1 import sys
2 sys.maxsize
```

Finden Sie heraus, was das bedeutet!

Das ergibt die Ausgabe

```
3 9223372036854775807
```

Aus Ganzzahlen werden beim Dividieren Floats, es sei denn, man verwendet `//` beim Dividieren:

```
4 >>> 42/10
5 4.2
6 >>> 42//10
7 4
8 >>>
```

Es wird bei der Ganzzahldivision nicht gerundet, sondern abgeschnitten.

5.2.6.2.2 float - Fließkommazahlen

Fließkommazahlen. Die sind zwar praktisch, aber nicht ungefährlich. Versuchen Sie mal Folgendes (auf einer Konsole):

```
1 0.1 + 0.2 == 0.3
```

5 Einführung in Python – Erste Schritte

Das ist keine unangenehme Eigenschaft der Programmiersprache, dieses Problem haben Sie in jeder Programmiersprache! Das ist ein Problem des Datenformats.

Um das zu verstehen, lassen Sie

```
0.1 + 0.2
```

ausführen. Das Ergebnis ist mit unausweichlichen Rundungsfehlern dieses Datenformats zu erklären.

Was also tun? Probieren Sie Folgendes:

```
>>> import math
>>> math.isclose( 0.1 + 0.2, 0.3)
True
>>> math.isclose( 0.1 + 0.2, 0.3, 1e-9)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: isclose() takes exactly 2 positional arguments (3 given)
>>> math.isclose( 0.1 + 0.2, 0.3, rel_tol=1e-9)
True
>>>
```

Tip: [The Right Way To Compare Floats in Python](#)

Was ist passiert?

Wir können uns Informationen anzeigen lassen über diesen Datentyp:

```
1 import sys
2 sys.float_info
```

Das ergibt die Ausgabe (umgebrochen und eingerückt für bessere Lesbarkeit):

```
3 sys.float_info( \
4     max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308, min=2.2250738585072014e-308, min_
exp=-1021, \
5     min_10_exp=-307, dig=15, mant_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1 \
6 )
```

5.2.6.2.3 String

```
1 s = "Ich bin eine Zeichenkette"
2 s = s.replace( "z", "Z")
```

Strings können eine ganze Menge oder - andersrum - Sie können eine ganze Menge anstellen mit Strings. Was alles, entnehmen Sie den Dokumenten, die Sie auf der [Website von Python](#) finden oder einfach mit

```
dir( str)
```

auf der Konsole

```
Python 3.10.4 (main, Mar 23 2022, 23:05:40) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> dir( str)
['_add_', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__',
'_format_', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__',
'_hash_', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__',
'_mod_', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
'_rmod_', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'ca-
pitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find',
'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdi-
git', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle',
'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix',
'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip',
'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper',
'zfill']
>>>
```

Python kennt 4 Arten von Anführungszeichen: ' , " , "" , "" "" .

Aufgabe

Finden Sie heraus, was die Unterschiede der 4 Möglichkeiten von Anführungszeichen sind und wo ihr Einsatz liegt!

5.2.6.2.4 Tuples

Tuples können ein oder mehrere Elemente enthalten. Sie können nicht verändert werden, wohl aber deren Elemente, wenn sie *mutable* sind (wie Listen).

Beispiele für Tuples:

```
1 t = ( 1, ) # Beachte das Komma!  
2 t = ( 0, 1, 2, "eins", "zwei", "drei", [ None, False, True])
```

Wir versuchen, ein Element zu ändern:

```
t[ 0] = "Hollodrio"
```

Das ergibt die Ausgabe

```
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_415212/1857332761.py in <module>  
----> 1 t[ 0] = "Hollodrio"  
  
TypeError: 'tuple' object does not support item assignment
```

Auch das funktioniert nicht:

```
t[ -1] = []
```

und bewirkt

```
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_415212/2145338212.py in <module>  
----> 1 t[ -1] = []  
  
TypeError: 'tuple' object does not support item assignment
```

Das hier allerdings funktioniert:

```
3 t[ -1][ 0] = "Hollodrio"  
1 t
```

und ergibt

```
(0, 1, 2, 'eins', 'zwei', 'drei', ['Hollodrio', False, True])
```

5.2.6.2.5 list - Listen

Listen sind wie [Tuple](#) Sequenztypen. Sie können 0 oder mehr Elemente enthalten. Listen können wie [Tuple](#) im Unterschied zu Arrays oder Listen in anderen Programmiersprachen auch Elemente unterschiedlicher Datentypen enthalten

```
L = [ *"Diese Liste enthält".split(), 6, "Strings und", 2, "Integer"]
```

Geben Sie das auf der Konsole ein und vergessen Sie dabei den '*' nicht!

```
>>> L = [ *"Diese Liste enthält".split(), 6, "Strings und", 2, "Integer"]
```

```
>>> L
['Diese', 'Liste', 'enthält', 6, 'Strings und', 2, 'Integer']
>>>
```

Was aber macht der '*'? Versuchen Sie das Ganze nochmals einzugeben, aber dieses Mal ohne den '*'!

```
>>> L = [ "Diese Liste enthält".split(), 6, "Strings und", 2, "Integer"]
>>> L
[['Diese', 'Liste', 'enthält'], 6, 'Strings und', 2, 'Integer']
>>>
```

Elemente von Listen sind über Ihren Index ansprechbar:

```
>>> L[ 0]
['Diese', 'Liste', 'enthält']
>>> L[ 0][ 0]
'Diese'
>>> L[ 1][ 0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'int' object is not subscriptable
>>> L[ 1]
6
>>>
```

Aufgabe

Interpretieren Sie obige Konsolen-Session, mitsamt dem Fehler!

Listen sind sehr vielseitig. Es können einzelne Elemente oder ganze Listen angehängt werden und - wie oben schon gescheen - elementweise darauf zugegriffen werden. Über Listen kann aber auch iteriert werden. In Listen kann auch gesucht werden, sie können kopiert und gelöscht werden usw..

Die Aufzählung ist nicht vollständig, schauen Sie für einen vollständigen Überblick in die Standard-Dokumentation von *Python* zum Thema Listen⁵.

5.2.6.2.6 Iterativer Zugriff auf Elemente einer Liste

```
for each in [ 1, 2, "drei", 4.0, None, {}]:
    print( each)
```

Die Bedeutung des letzten Elementes in der Liste, über die hier iteriert wird, finden Sie unter [Dictionaries](#)

5.2.6.2.7 List Comprehensions

Ein Statement für die leere Liste und eine `for`-Schleife, um Listen zu erzeugen? Das geht mit [List Comprehensions](#) (s. auch die [Python-Dokumentation dazu](#)), die Listen in einer Zeile (oder als Teil einer *on the fly*) erzeugen. So kann

```
1 L = []
2 for i in range( 42):
3     L.append( i)
```

kürzer als

```
4 L = [ i for i in range( 42)]
```

5 Z.B. [hier](#) und [hier](#). Gehen Sie jeweils eine Ebene höher und suchen Sie nach `list`. Stöbern Sie!

geschrieben werden. Da sind natürlich auch Expressions möglich:

```
5 quadrate_bis_und_mit_42 = [ i*i for i in range( 42)]
```

Aber auch Bedingungen können eingearbeitet werden:

```
6 quadrate_bis_und_mit_42_ohne_die_durch_10_teilbaren = [ i*i for i in range( 42) if i % 10]
```

100, 400, ... werden in dieser Liste also nicht enthalten sein.

```
7 durch_10_teilbare_ganzzahlen = [x for x in range( random.randint( 100, 1000)) if x%10 == 0]
```

5.2.6.2.8 dict - Dictionaries

Dictionaries sind wie Listen, nur dass die Elemente nicht über einen Index sondern über einen Schlüssel erreichbar sind (sie werden daher auch als *assoziative Datentypen* bezeichnet):

```
>>> Max = { "Name": "Mustermann", "Alter": 13, "Geschwister": [ "Anton", "Egon"] }
>>> Max
{'Name': 'Mustermann', 'Alter': 13, 'Geschwister': ['Anton', 'Egon']}
>>> Max[ "Name"]
'Mustermann'
>>> Max[ "Alter"]
13
>>> Max[ "Geschwister"]
['Anton', 'Egon']
>>>
```

Ein Beispiel aus der Technik:

```
>>> irs = { "Typ": "Entfernungsmesser", "Prinzip": "Infrarotsensor", "Hersteller": "Sharp", "Modell": None, "Maximaldistanz": 0.150}
>>> irs
{'Typ': 'Entfernungsmesser', 'Prinzip': 'Infrarotsensor', 'Hersteller': 'Sharp', 'Modell': None, 'Maximaldistanz': 0.15}
>>>
>>> irs[ "Maximaldistanz"]
0.15
>>> irs[ "Maximale Distanz"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Maximale Distanz'
>>>
```

Der Schlüssel "Maximaldistanz" existiert nicht in diesem Dictionary, weshalb eine `KeyError`-Exception ausgelöst wird.

Andere Namen sind auch *Assoziatives Array*, *Hash* oder *Map*.

5.2.6.2.9 Sets (Mengen)

Wir erzeugen ein Set (eine Menge) so:

```
1 menge = set()
```

`set` ist eine Klasse und mit `set()` führen Sie deren [Ctor](#) aus.

Sets können, was man von Mengen so erwartet:

- Kein Element mehr als 1-mal aufnehmen

5 Einführung in Python – Erste Schritte

- Vereinigungen bilden
- Durchschnitte bilden

usw.

Aufgabe

Picken Sie sich eine Methode heraus und probieren Sie sie aus?

Ein `dir( set )` oder auch ein `help( set )` schaffen Klarheit.

5.2.6.2.10 Files

Files schreiben:

```
1 pathname = "snapshot_of_the_rovers_propulsion.txt"
2 etixL = 42
3 etixR = 4242
4 f = open( pathname, "w")
5 f.write( "Encoder-Tix\t%d\t%d" % (etixL, etixR))
6 f.close()
7 print( "File '%s' ist geschrieben." % pathname)
```

Es gibt Situationen, die verhindern, dass das File geschlossen wird. Das kann man verhindern, indem man das File als [Context-Manager](#) ausführt:

```
1 etixL = 42
2 etixR = 4242
3 with open( "2delete.txt", "w") as f:
4     f.write( "Encoder-Tix\t%d\t%d" % (etixL, etixR))
5
6 print( "File '%s' ist geschrieben." % pathname)
```

Außerdem machen [Context-Manager](#) den Code übersichtlicher.

Files lesen:

```
1 with open( pathname, "r") as f:
2     print( f.read())
```

5.2.6.2.11 Klassen

Klassen beginnen mit dem Schlüsselwort `class`. Die weitere Diskussion verschieben wir an dieser Stelle aber besser, bis wir uns über [objektorientierte Programmierung](#) unterhalten haben.

5.2.6.3 Ausdrücke und Anweisungen

Was ist der Unterschied?
Finden Sie's raus!

Ausdrücke und Anweisungen oder englisch *Expressions and Statements*.

5.2.6.4 Steuerung des Programmflusses

Welche Teile Ihres Programms durchlaufen werden und wie oft, steuern Sie mit

Finden Sie heraus, was
`pass` bedeutet!

if-else

```
1 if some_condition():
2     pass # some_condition() is fulfilled
3
4 else:
5     pass # some_condition() is NOT fulfilled
```

while

```
6 while some_condition():
7     pass # Execute this block as long as some_condition() is true.
```

for

```
8 for i in range( 42):
9     pass # Do this 42 times
10
11 for c in ( 'a', 'b', 'c'):
12     pass # Do this for 'a', 'b', and 'c'.
```

Und weil man

```
13 cc = ( 'a', 'b', 'c')
14 for i in range( len( cc)):
15     print( "Character %d == %s." % (i, cc[ i]))
```

so soft braucht, kann man das kürzer schreiben:

```
16 cc = ( 'a', 'b', 'c')
17 for i, c in enumerate( cc):
18     print( "Character %d == %s." % (i, c))
```

5.2.6.5 Programmierstil

Überspringen Sie dieses Kap. beim ersten Durcharbeiten und kommen Sie zurück, wenn Sie die ersten Schritte in Objekt-orientierter Programmierung gemacht haben.

Programmierrichtlinien sollen einen einheitlichen Stil in der Programmierung forcieren⁶, damit der Code jedes Mitarbeiters gleich gut lesbar ist. Da wird dann definiert, wie viele Leerzeichen wo hineingehören, wo geschwungene Klammern zu stehen haben ([Python](https://www.python.org/) hat - Gott sei Dank - keine) usw. usf. Oft kommen wichtige Aspekte eines guten Programmierstils dadurch zu kurz, Aspekte, die's wert wären, dass sie in einer Programmierrichtlinie erwähnt werden. Und weil das so ist ober eben nicht, sind Programmierrichtlinien immer wieder Ursache von regelrechten Glaubenskriegen. Der Grund ist ein einfacher: Hier kann jeder, wirklich jeder mitreden („Kein Abstand nach einer Argumentenliste einleitenden Klammer oder genau einer?“).

Wenn Sie in einer größeren SW-Abteilung arbeiten, dann haben Sie vermutlich keine Wahl und müssen einem definierten Stil folgen. Das ist meist gar nicht so schlecht oder rettet sogar Menschenleben⁷. Apropos *Menschenleben*: Führen Sie sich in einer ruhigen Minute <https://de.wikipedia.org/wiki/Therac-25> (oder <https://blog.grio.com/2021/07/when-small-software-bugs-cause-big-problems.html>) zu Gemüte⁸.

Wir schlagen daher folgendes vor: Finden Sie Ihren eigenen Stil, wenn Sie sich das leisten können (weil Sie alleine an einem Projekt arbeiten). Hierzu ein paar Anregungen:

1. Kommentare: Kommentieren Sie Umstände, die aus dem Code nicht ersichtlich sind. Kommentieren Sie vornehmlich das *Warum* und weniger das *Was* und schon gar nicht Offensichtliches. Das hier ist daher nicht hilfreich:

```
i += 1 # i inkrementieren
```

6 Siehe z.B. <https://de.wikipedia.org/wiki/Programmierstil>, <https://ssw.jku.at/Teaching/Lectures/PI2/2019/Guidelines.html> (wo der Programmierstil in die Prüfungsbewertung eingeht), <https://www.embedded-software-engineering.de/coding-guidelines-sind-programmierrichtlinien-fluch-oder-segen-a-795748/>, https://www.microconsult.de/blog/2019/01/fl_programmierrichtlinien/ (launige Gedanken zu Programmierrichtlinien; nur mit Jodas Grammatik hat's der Autor dort nicht so ;-)).

7 <https://de.wikipedia.org/wiki/MISRA-C>

8 Für starke Nerven: Bilder, die zeigen, was dieses System *unter bestimmten Umständen* angerichtet hat, finden Sie mittels Bildersuche nach *therac-25 burns*.

2. Keine zu tiefen Klassenhierarchien, [denn diese Tiefe macht Sie unbeweglicher](#)⁹, je tiefer sie geht.
3. Private Attribute beginnen mit zwei Unterstrichen.
4. Methoden, die privat oder protected sind, beginnen und enden mit einem Unterstrich.
5. Methodennamen beginnen mit Kleinbuchstaben.
6. Variablennamen – *CamelCase* (Pascal-Stil) oder *snake_case*? Das geht am eigentlichen Problem der Namenssystematik vorbei. Ein Beispiel: Wollen Sie Ihre Variable „Anzahl Studenten“
 1. anzahl_studenten
 2. studenten_anzahl
 3. studentenanzahl

nennen? Antwort: Alle drei Varianten sind ok., je nachdem wie das große Bild aussieht.

(1) liegt auf der Hand, immer.

(2) liegt nahe, wenn man auch `studenten_lven`, `studenten_kohortename` usw. hat.

(3) wenn (2) nicht zutrifft; Zusammenschreiben – auch englischer Variablen oder Methodennamen - immer dann, wenn man auch im Deutsch ein Wort daraus machen könnte.
7. Nach öffnenden Klammern gerne ein Leerzeichen.
8. Globale Daten beginnen mit einem Unterstrich gefolgt von einem Großbuchstaben.

5.2.7 Idioms

Idiom, was ist ein Idiom in einer oder für eine Programmiersprache? [Wikipedia](#) weiß hierzu: "Idiome können als Umsetzungen (Implementierungen) von abstrakten Mustern in einer spezifischen Programmiersprache verstanden werden. Also wie einfache Aufgaben (niedriger Abstraktionsstufe) in einer Programmiersprache gelöst werden. (...) Ein Idiom zeichnet sich somit durch folgende Eigenschaften aus:

- es ist programmiersprachenspezifisch
- es ist zu finden in Feinentwurf und Implementierung (niedrige Abstraktionsebene).

Es geht also dabei um die Implementierung von speziellen Entwurfsaspekten.

[\[https://de.wikipedia.org/wiki/Idiom_\(Softwaretechnik\)\]](https://de.wikipedia.org/wiki/Idiom_(Softwaretechnik))"

Hier ein paar Idioms, die sich als hilfreich erweisen könnten.

5.2.7.1 Werte tauschen

```
1 x, y = 42, 24
2 print( "Before swap:", x, y)
3 x, y = y, x
4 print( "After swap:", x, y)
```

⁹ "The fragile base class problem is a fundamental architectural problem of object-oriented programming systems where base classes (superclasses) are considered "fragile" because seemingly safe modifications to a base class, when inherited by the derived classes, may cause the derived classes to malfunction. The programmer cannot determine whether a base class change is safe simply by examining in isolation the methods of the base class. [https://www.wikiwand.com/en/Fragile_base_class]"

5.2.7.2 Aus einer oder mehreren Listen mach' eine Liste

```
1 import itertools
2
3 regular_list = [[1, 2, 3, 4], [5, 6, 7], [8, 9, 10]]
4 flat_list = list(itertools.chain(*regular_list))
5
6 print('Original list', regular_list)
7 print('Transformed list', flat_list)
```

5.2.7.3 Aus zwei Listen mach' (durch Verschränken) eine Liste von Paaren

```
1 list_of_tuples = list(zip([1, 2, 3], [4, 5, 6, 7]))
2 print("List of tuples created from 2 lists:", list_of_tuples)
```

5.2.7.4 Aus einer Liste von Paaren mach' zwei Listen

```
1 list(zip(*[(1, 2), (3, 4), (5, 6)]))
```

5.2.7.5 Zugriff auf Listenelemente mittels negativer Indices

Sehr praktisch generell, im Besonderen aber, wenn man mit Elementen zu hantieren hat wie in der Regelungstechnik, bspw. x_{k-1} :

```
1 x = [1, 2, 3]
2 x[0] # Aktueller Wert
3 x[-1] # Wert des letzten Laufs
4 x[-2] # Wert des vorletzten Laufs
5 x[-3] # Semantischer Fehler: Ist x[0]!
6
7 print(x[0], x[-1], x[-2], x[-3]).
```

5.2.7.6 Daten aus/in Objekte/n lesen/schreiben

Java-Programmierer verwenden sogenannte Getter und Setter, um auf Daten eines Objektes zuzugreifen. C++-Programmierer können entsprechende Zugriffsmethoden überladen (könnten Java-Programmierer auch), denn wer will schon statt `sin(phi)` `get_sin(phi)` schreiben (müssen)?!

Wie können wir das in Python lösen? Getter und Setter sind umständlich und hässlich, Überladen geht wegen der dynamischen Natur von Python nicht. Eine Möglichkeit wären Properties; es sei hier allerdings dem Leser überlassen, sich diesbezüglich schlau zu machen.

An dieser Stelle soll folgendes Python-Idiom für den Zugriff auf Objektdaten (meistens eine schlechte Idee übrigens, oft aber notwendig) gezeigt werden¹⁰:

```
1 class Parameter:
2
3     def __init__(self, defaultvalue=0.0):
4         self.__value = defaultvalue
5
6     def value(self, arg=None):
7         if arg is None:
8             return self.__value
```

¹⁰ Noch können Sie vermutlich wenig mit diesem Code anfangen. Das wird sich ändern, wenn Sie das Kapitel [OOP](#) durchgearbeitet haben werden. Kommen Sie dann wieder hierher zurück!

5 Einführung in Python – Erste Schritte

```
9
10     self.__value = arg
11     return self
```

Bei einer solchen Klasse liegt es auf der Hand, einen Accessor zu realisieren, der den Wert eines Parameters liest/schreibt, wozu sollte ein Parameter sonst gut sein.

Allerdings sollte der Einsatz von Accessors gut überlegt sein, stellen sie doch einen gewissen Widerspruch zur Forderung der Datenkapselung dar: Nehmen Sie an, Sie wollten den Wert ausdrucken. Wahrscheinlich würden Sie den Wert lesen und dann einem Drucker übergeben. Eine hinsichtlich Datenkapselung elegantere Methode wäre allerdings, den Parameter seinen Wert selber drucken zu lassen:

```
1 def print_value( self, printer: callable):
2     printer( self.value())
```

6 Grundlagen

6.1 Variable und Datentypen bzw. -strukturen

6.1.1 Variable

Eine Variable ist ein Speicherbereich einer bestimmten Länge, gemessen in Bytes (= 8 Bits), der Daten enthält. Speicherbereiche werden über Adressen angesprochen. In höheren Programmiersprachen braucht man nicht mit Speicheradressen zu hantieren, sondern kann diesen Speicherbereichen Namen geben. Solche Namen könnte man auch – und andere Programmiersprachen wie C und C++ tun das – als Pointer bezeichnen. Die Adressen der Speicherbereiche, auf die die Namen "zeigen", können wir uns mit der Funktion `id()` anzeigen lassen.

Höhere Programmiersprachen verbinden mit solchen Speicherbereichen einen Typ wie Integer oder Float. In statisch typisierten Programmiersprachen muss einer Variablen ein Typ fest zugeordnet werden. In dynamisch typisierten Sprachen ist das nicht notwendig, weil Namen einfach nur eine Art Labels sind, die an Speicherbereiche gebunden werden können, eben auf sie zeigen.

Rechner raus und – machen!

Lassen Sie uns das gemeinsam „erforschen“:

```
>>> i = 42
>>> id( i)
139791546582544
>>> i = 43
>>> id( i)
139791546582576
>>> j = i
>>> id( i)
139791546582576
>>> id( j)
139791546582576
>>>
```

Erklärungen?

Folgende Abbildung mag hilfreich sein.

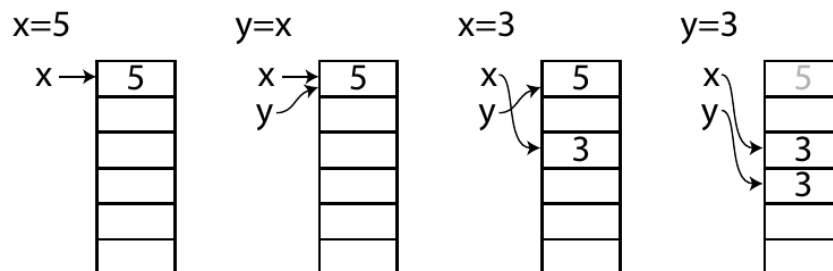


Fig. 1: Quelle: AYARS: Computational Physics with Python. 2013

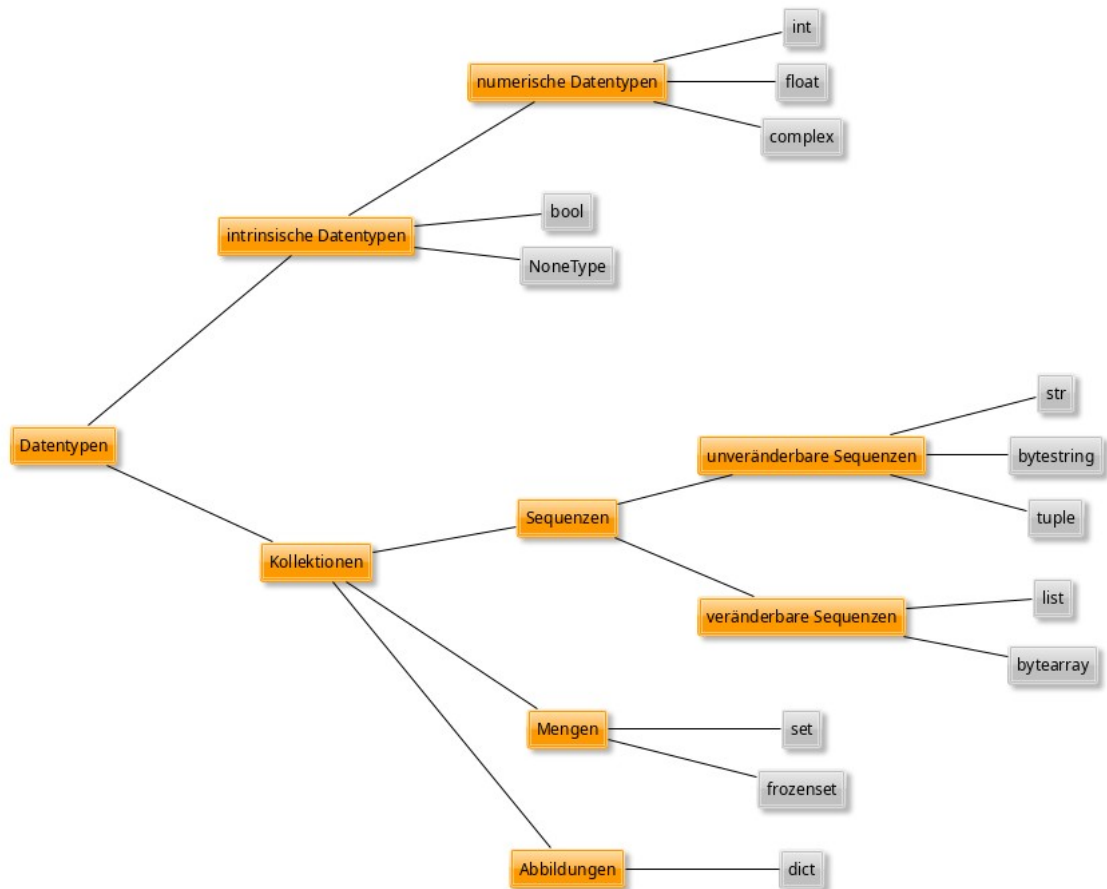


Fig. 2

6.1.2 Numerische Datentypen

Hierzu gehören die Datentypen

- Ganzzahl (`int`)
- Fließkommazahl (`float`)
- Boolean (`bool`) im weiteren Sinne

6.1.2.1 `int` – Ganzzahlen

Ganzzahlen, hierzu gibt's nicht viel zu sagen, außer - und das erscheint mir nicht unwichtig - dass Ganzzahlen in Python nicht überlaufen können:

```

>>> 42**42
150130937545296572356771972164254457814047970568738777235893533016064
>>> type( _)
<class 'int'>
>>>
  
```

Wir können vom System Informationen über diesen Datentyp erhalten:

6 Grundlagen

```
import sys
sys.maxsize
```

Finden Sie heraus, was das bedeutet!

Das ergibt die Ausgabe

```
9223372036854775807
```

Aus Ganzzahlen werden beim Dividieren Floats, es sei denn, man verwendet `//` beim Dividieren:

```
>>> 42/10
4.2
>>> 42//10
4
>>>
```

Es wird bei der Ganzzahldivision nicht gerundet, sondern abgeschnitten.

6.1.2.2 float – Fließkommazahlen

Fließkommazahlen. Die sind zwar praktisch, aber nicht ungefährlich. Versuchen Sie mal Folgendes (auf einer Konsole):

```
0.1 + 0.2 == 0.3
```

Um das zu verstehen, lassen Sie

```
0.1 + 0.2
```

ausführen. Das Ergebnis ist mit unausweichlichen Rundungsfehlern dieses Datenformats zu erklären.

Was also tun? Probieren Sie Folgendes:

```
>>> import math
>>> math.isclose( 0.1 + 0.2, 0.3)
True
>>> math.isclose( 0.1 + 0.2, 0.3, 1e-9)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: isclose() takes exactly 2 positional arguments (3 given)
>>> math.isclose( 0.1 + 0.2, 0.3, rel_tol=1e-9)
True
>>>
```

Wir können uns Informationen anzeigen lassen über diesen Datentyp:

```
import sys
sys.float_info
```

Das ergibt die Ausgabe (umgebrochen und eingerückt für bessere Lesbarkeit):

```
sys.float_info( \
    max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308,
    min=2.2250738585072014e-308, min_exp=-1021, \
    min_10_exp=-307, dig=15, mant_dig=53, epsilon=2.220446049250313e-16, ra-
    dix=2, rounds=1 \
)
```

6.1.2.3 complex - Komplexe Zahlen

Die Beschäftigung mit diesem Datentyp ist eine gute Gelegenheit, einen Blick in die wirklich

Das ist keine unange-
nehme Eigenschaft der
Programmiersprache,
dieses Problem haben
Sie in jeder Program-
miersprache! Das ist ein
Problem des Datenfor-
mats.

Tip: [The Right Way To Compare Floats in Python](#)

sehr(!) gute [Python-Dokumentation](#) zu werfen.

Ein Klick auf den [Global Module Index] liefert die Module, die mit Python mitkommen. Wir suchen nach `complex` und finden das der Klasse `complex`. Ein Klick auf [Numeric Types — int, float, complex](#) zeigt uns, dass die Klasse `complex` nicht gerade ein umfangreiches Interface hat.

Dann halt so: Wir geben `complex` ein in die *Quick Search* und finden einen Eintrag für `complex` als ersten Treffer und [cmath - Mathematical functions for complex numbers](#). Letzterem gehen wir nach und siehe da, lauter praktische Funktionen. Probieren wir mal die Konversion *kartesisch nach polar* und zurück:

```
from math import *
import cmath

>>> z = complex( 1, 1)
>>> z
(1+1j)
>>>
>>> r, phi = cmath.polar( z)
>>> r
1.4142135623730951
>>> degrees( phi)
45.0
>>> assert z == cmath.rect( r, phi)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError
>>>
```

Das kennen wir schon von den Fließkommazahlen. Wir suchen und finden die Bestätigung:

```
>>> z, cmath.rect( r, phi)
((1+1j), (1.0000000000000002+1j))
>>>
```

Wir müssen also

```
>>> cmath.isclose( z, cmath.rect( r, phi))
True
>>>
```

schreiben, damit der Test klappt.

6.1.3 Sequentielle Datentypen

Hierzu gehören

- Strings
- Tupel
- Listen.

6.1.3.1 str - Strings (Zeichenketten)

Strings sind Zeichenketten. Seit *Python3* sind die Zeichen [Unicode](#)s.

Das String-Processing ist in Python wirklich stark und es liegt manchmal sogar nahe, Zahlenwerte, die irgendwie zu verändern sind, in Strings zu konvertieren, die Änderung vorzunehmen und

6 Grundlagen

dann wieder in einen Zahlenwert zu verwandeln.

```
1 s = "Ich bin eine zeichenkette"
2 s = s.replace("z", "Z")
```

Strings können eine ganze Menge oder - andersrum - Sie können eine ganze Menge anstellen mit Strings. Was alles, entnehmen Sie den Dokumenten, die Sie auf der Website von [py] finden oder einfach mit

```
dir( str)
```

auf der Konsole

```
Python 3.10.4 (main, Mar 23 2022, 23:05:40) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> dir( str)
['_add_', '_class_', '_contains_', '_delattr_', '_dir_', '_doc_',
'_eq_', '_format_', '_ge_', '_getattr_', '_getitem_', '_getne-
wargs_', '_gt_', '_hash_', '_init_', '_init_subclass_', '_iter_',
'_le_', '_len_', '_lt_', '_mod_', '_mul_', '_ne_', '_new_',
'_reduce_', '_reduce_ex_', '_repr_', '_rmod_', '_rmul_', '_setat-
tr_', '_sizeof_', '_str_', '_subclasshook_', 'capitalize', 'casefold',
'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'for-
mat_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit',
'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle',
'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition',
'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpar-
tition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip',
'swapcase', 'title', 'translate', 'upper', 'zfill']
>>>
```

Finden Sie heraus, was die Unterschiede der 4 Möglichkeiten von Anführungszeichen sind und wo ihr Einsatz liegt!

Python kennt übrigens 4 Arten von Anführungszeichen: ' ', " ", '"""', '"""'.

Wenn wir bspw. 2 Strings in Großbuchstaben umwandeln und dann zu einem durch einen Bindestrich getrennten Wort verschmelzen wollen, könnten wir schreiben:

```
1 worte = ("String", "Methoden")
2 wort = "-".join( worte)
3 print( wort)
```

Die beiden Worte haben wir als Tuple organisiert. Diesen Datentyp werden wir in einem der nächsten Kapitel kennenlernen.

Die Trennung zurück kann erfolgen mit

```
worte = wort.split( "-")
print( worte)
```

Geliefert werden die beiden ursprünglichen Worte, aber in einer Liste. Auch diesen Datentyp werden wir in einem der nächsten Kapitel kennenlernen.

6.1.3.2 list – Listen

Listen sind wie [Tuple](#) Sequenztypen. Sie können 0 oder mehr Elemente enthalten. Listen können wie [Tuple](#) im Unterschied zu Arrays oder Listen in anderen Programmiersprachen auch Elemente unterschiedlicher Datentypen enthalten

```
L = [ "Diese Liste enthält".split(), 6, "Strings und", 2, "Integer"]
```

Geben Sie das auf der Konsole ein und vergessen Sie dabei den '*' nicht!

```
>>> L = [ "Diese Liste enthält".split(), 6, "Strings und", 2, "Integer"]
```

```
>>> L
['Diese', 'Liste', 'enthält', 6, 'Strings und', 2, 'Integer']
>>>
```

Was aber macht der '*'? Versuchen Sie das Ganze nochmals einzugeben, aber dieses Mal ohne den '*'!

```
>>> L = [ "Diese Liste enthält".split(), 6, "Strings und", 2, "Integer"]
>>> L
[['Diese', 'Liste', 'enthält'], 6, 'Strings und', 2, 'Integer']
>>>
```

Elemente von Listen sind über Ihren Index ansprechbar:

```
>>> L[ 0]
['Diese', 'Liste', 'enthält']
>>> L[ 0][ 0]
'Diese'
>>> L[ 1][ 0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'int' object is not subscriptable
>>> L[ 1]
6
>>>
```

Interpretieren Sie obige Konsolen-Session, mit-samt dem Fehler!

Listen sind sehr vielseitig. Es können einzelne Elemente oder ganze Listen angehängt werden und - wie oben schon geschehen - elementweise darauf zugegriffen werden. Über Listen kann aber auch iteriert werden. In Listen kann auch gesucht werden, sie können kopiert und gelöscht werden usw..

Die Aufzählung ist nicht vollständig, schauen Sie für einen vollständigen Überblick in die Standard-Dokumentation von *Python* zum Thema Listen¹¹.

6.1.3.2.1 Iterativer Zugriff auf Elemente einer Liste

```
1 for each in [ 1, 2, "drei", 4.0, None, {}]:
2     print( each)
```

Die Bedeutung des letzten Elementes in der Liste, über die hier iteriert wird, finden Sie unter [Dictionaries](#).

6.1.3.2.2 List Comprehensions

Ein Statement für die leere Liste und eine `for`-Schleife, um Listen zu erzeugen? Das geht mit [List Comprehensions](#) (s. auch die [Python-Dokumentation dazu](#)), die Listen in einer Zeile (oder als Teil einer *on the fly*) erzeugen. So kann

```
1 L = []
2 for i in range( 42):
3     L.append( i)
```

kürzer als

```
L = [ i for i in range( 42)]
```

geschrieben werden. Da sind natürlich auch Expressions möglich:

```
quadrade_bis_und_mit_42 = [ i*i for i in range( 42)]
```

¹¹ Z.B. [hier](#) und [hier](#). Gehen Sie jeweils eine Ebene höher und suchen Sie nach `list`. Stöbern Sie!

6 Grundlagen

Aber auch Bedingungen können eingearbeitet werden:

```
quadrade_bis_und_mit_42_ohne_die_durch_10_teilbaren = [ i*i for i in
range( 42) if i % 10]
```

100, 400, ... werden in dieser Liste also nicht enthalten sein.

```
durch_10_teilbare_ganzzahlen = [x for x in range( random.randint( 100, 1000))
if x%
```

6.1.3.3 tuple – Tuples

Was ist eigentlich der Unterschied zwischen Listen und Tuples?

Finden Sie's heraus! Auf der Seite [Python Tuples vs. Lists: When to Use Tuples Instead of Lists](#) ist das sehr übersichtlich beschrieben. Schauen Sie rein!

Tuples können ein oder mehrere Elemente enthalten. Sie können nicht verändert werden, wohl aber deren Elemente, wenn sie *mutable* sind (wie Listen).

Beispiele für Tuples:

```
t = ( 1,) # Beachte das Komma!
t = ( 0, 1, 2, "eins", "zwei", "drei", [ None, False, True])
```

Wir versuchen, ein Element zu ändern:

```
t[ 0] = "Hollodrio"
```

Das ergibt die Ausgabe

```
TypeError                                Traceback (most recent call last)
```

```
/tmp/ipykernel_415212/1857332761.py in <module>
----> 1 t[ 0] = "Hollodrio"
```

```
TypeError: 'tuple' object does not support item assignment
```

Auch das funktioniert nicht:

```
t[ -1] = []
```

und bewirkt

```
TypeError                                Traceback (most recent call last)
```

```
/tmp/ipykernel_415212/2145338212.py in <module>
----> 1 t[ -1] = []
```

```
TypeError: 'tuple' object does not support item assignment
```

Das hier allerdings funktioniert:

```
t[ -1][ 0] = "Hollodrio"
t
```

und ergibt

```
(0, 1, 2, 'eins', 'zwei', 'drei', ['Hollodrio', False, True])
```

6.1.3.4 Rechnen mit Sequenztypen?

6.1.3.4.1 Strings

Mit Strings kann man nicht rechnen, auch wenn es auf den ersten Blick so aussieht.

Addition

Sinnvollerweise hängt der +-Operator Strings aneinander. So gibt

```
print( "Hello" + " " + "World" + "!" )
```

die Zeichenkette "Hello World!" aus.

Multiplikation

Der *-Operator vervielfältigt Zeichenketten (die auch nur 1 Zeichen lang sein können wie im nächsten Beispiel):

```
s = "Hello" + " " + "World" + "!"
print( s )
print( "=" * len( s ) )
```

Die Ausgabe ist die gleich wie oben, nur dass sie jetzt unterstrichen ist.

6.1.3.4.2 Listen

Wie sieht das mit Listen oder Tuples aus? Vektoren können durch Listen oder Tuples interpretiert werden. Rechnen die arithmetischen Operatoren vllt hier?

Das Ergebnis von

```
print( [1, 2, 3] + [4, 5, 6] )
```

ist dem bei den Strings äquivalent:

```
[1, 2, 3, 4, 5, 6]
```

Ebenso die Multiplikation:

```
print( [ 42 ] * 13 )
```

6.1.3.4.3 Tuples

Das gerade Gezeigte gilt auch für Tuples.

6.1.3.4.4 Rechnen mit Listen und Tuples

Überspringen Sie dieses Kap., bis Sie Kap. 7 durchgearbeitet haben.

Wollte man das ändern, könnte man - hier greife ich [objektorientiertem Programmieren mit Python](#) vor - folgende Klasse als Subclass von `list` definieren:

```
1 class V3( list ):
2
3     def __add__( self, other ):
4         return self.__class__( [ L+R for L, R in zip( self, other ) ] )
5
6 r = V3( [1, 2, 3] ) + V3( [4, 5, 6] )
7 print( r )
```

Wenn wir dem Contructor keine Listen übergeben wollen, sondern einfach eine Sequenz von Zahlenwerten, nämlich

```
V3( 1, 2, 3 )
```

dann müssen wir auch den Ctor überschreiben:

```
1 class V3( list ):
2
3     def __init__( klass, *args ):
4         return super().__init__( args )
5
6     def __add__( self, other ):
7         return self.__class__( *[ L+R for L, R in zip( self, other ) ] )
```

6 Grundlagen

```

8
9 r = V3( 1, 2, 3) + V3( 4, 5, 6)
10 print( r)

```

Wollten wir Letzteres bei der Ableitung von `tuple` erreichen, müssen wir anders vorgehen, weil Tuples - das haben Sie mittlerweile sicher herausgefunden - *readonly* sind.

Was ist der Unterschied zwischen `__new__` und `__init__`?

Versuchen Sie's herauszufinden, z. B. hier:

[Python Tuples vs. Lists: When to Use Tuples Instead of Lists](#)

Hier müssen wir in die Konstruktion des Objekts (das dann erst mit `__init__` initialisiert wird) eingreifen:

```

1 class V3( tuple):
2
3     def __new__( klass, *args):
4         return tuple.__new__( klass, args)
5
6     def __add__( self, other):
7         return Tuple( *[ L+R for L, R in zip( self, other)])
8
9 r = V3( 1, 2, 3) + V3( 4, 5, 6)
10 print( r)

```

In beiden Fällen wird als Ergebnis der Summenvektor angezeigt. Python führt hierzu die *Magic Method* (oder auch *Special Method*) `__repr__` aus. Da wir keine definiert haben, wird jene der Base Class ausgeführt. Natürlich können wir das durch Overriding auch ändern, was durchaus sinnvoll ist, weil `__repr__` eigwntlich dafür gedacht ist, ein Objekt so als String darzustellen, dass es aus diesem String per `eval` wieder erzeugt werden könnte.

```

def __repr__( self):
    return "%s( %f, %f, %f)" % (self.__class__.__name__, *self[ :3])

```

Mit `self[ :3]` stellen wir sicher, dass nur 3 Elemente verwendet werden, falls der Vektor länger ist. Ist er kürzer, wird dieser Code nicht laufen.

Aber warum sollte `V3` nicht 3 Elemente enthalten? Antwort: Wir haben es offen gelassen! Folgende Definition würde dieses Problem beheben können:

```

1 class V3( tuple):
2
3     def __new__( klass, x, y, z):
4         return tuple.__new__( klass, (x, y, z))
5
6     def __add__( self, other):
7         return V3( *[ L+R for L, R in zip( self, other)])
8
9     def __repr__( self):
10         return "%s( %f, %f, %f)" % (self.__class__.__name__, *self)
11
12 r = V3( 1, 2, 3) + V3( 4, 5, 6)
13 print( r)

```

Probieren wir's aus:

```

r = V3( 1, 2, 3, 4) + V3( 4, 5, 6)
print( r)
Traceback (most recent call last):
  File "/home/fgeiger/atlantisl/lab/python/subclassing_tuple.py", line
12, in <module>
    r = V3( 1, 2, 3, 4) + V3( 4, 5, 6)
      ^^^^^^^^^^^^^^^^^
TypeError: V3.__new__() takes 4 positional arguments but 5 were given

```

Fällt Ihnen etwas auf an der Fehlermeldung? `V3` erwartet 3 Argumente, nicht 4. Und wir haben 4

Argumente übergeben, nicht 5. Der Grund ist, Python zählt den "Instanzen-Pointer" `self` mit!

6.1.4 Indexing, Slicing und weitere Operationen auf sequentielle Datentypen

Das Indizieren von Elementen sequentieller Datentypen funktioniert wie bei anderen Programmiersprachen auch. Praktisch hier allerdings, dass man auch "von hinten her" indizieren kann:

```
L = [ 0, 1, 2, 3]
print( L[ -1])
```

gibt eine 3 aus.

```
L = [ 0, 1, 2, 3]
print( L[ :-1])
```

gibt [0, 1, 2] aus und

```
L = [ 0, 1, 2, 3]
print( L[ 1:3])
```

gibt [1, 2] aus.

Das funktioniert ebenso mit Strings und Tuples.

Verändern kann man allerdings nur Listen, sei es durch schreibenden Zugriff auf Listenelemente oder durch Erweitern oder Löschen. Dazu ein Ausschnitt aus einer Konsolen-Session:

```
>>> L = []
>>> id( L)
140109184798144
>>> L.append( 1)
>>> L
[1]
>>> id( L)
140109184798144
>>> L.extend( [1,2,3])
>>> L
[1, 1, 2, 3]
>>> id( L)
140109184798144
>>>
```

Es ist - wie man sieht - immer die gleiche Liste.

Wie aber ist das mit einem String?

```
>>> s = "Elektronik"
>>> s
'Elektronik'
>>> id( s)
140109188612784
>>> s += " und Informationstechnologie Dual"
>>> s
'Elektronik und Informationstechnologie Dual'
>>> id( s)
140109186778384
>>>
```

Es handelt sich jeweils um einen anderen String! Strings sind *immutable*. Was also macht Python? Es nimmt den einen String und den anderen, fügt sie zusammen und steckt das Ergebnis in einen neuen Speicherbereich. Eine interessante Übung in C übrigens, aber das ist eine ande-

re ;-)

Sie werden sehen, dass es in Ihrer späteren Praxis nicht unwichtig ist, über den Zusammenhang von Name-Binding, Speicherbereichen und Mutability Bescheid zu wissen.

Dazu eine Python-Konsolen-Session:

```
>>> L = [1, 2, 3]
>>> L; id( L)
[1, 2, 3]
140109184794368
>>> LL = [4, 5, 6, L]
>>> LL; id( LL)
[4, 5, 6, [1, 2, 3]]
140109184799296
>>> L = [11, 22, 33]
>>> L; id( L)
[11, 22, 33]
140109184798144
>>> LL; id( LL)
[4, 5, 6, [1, 2, 3]]
140109184799296
>>>
```

haben Sie eine Erklärung für die drittletzte Zeile `[4, 5, 6, [1, 2, 3]]`?

Vermutlich haben Sie erwartet, dass da `[4, 5, 6, [11, 22, 33]]` steht. Nun, das können wir erreichen, indem wir schreiben:

```
>>> LL[ 3][ : ] = L
>>> LL; id( LL)
[4, 5, 6, [11, 22, 33]]
140109184799296
>>>
```

`L[ : ] = ...` **verändert** also die Liste *in place*. Der Vollständigkeit halber:

```
>>> LLL = L[ : ]
>>> LLL; id( LLL)
[11, 22, 33]
140109184795520
>>>
```

erzeugt eine Kopie einer bestehenden Liste und weist dieser einen neuen Speicherbereich zu.

Kopien beliebiger Objekte - shallow und deep - erstellen Sie übrigens mit den Methoden des Moduls `copy`.

6.1.5 Assoziative Datentypen

Hierzu gehören

- Dictionaries
- (Sets im weiteren Sinne)

6.1.5.1 dict – Dictionaries

Dictionaries sind wie Listen, nur dass die Elemente nicht über einen Index sondern über einen Schlüssel erreichbar sind (sie werden daher auch als *assoziative Datentypen* bezeichnet):

```
>>> Max = { "Name": "Mustermann", "Alter": 13, "Geschwister": [ "Anton",
"Egon"]}
>>> Max
{'Name': 'Mustermann', 'Alter': 13, 'Geschwister': ['Anton', 'Egon']}
>>> Max[ "Name"]
'Mustermann'
>>> Max[ "Alter"]
13
>>> Max[ "Geschwister"]
['Anton', 'Egon']
>>>
```

Ein Beispiel aus der Technik:

```
>>> irs = { "Typ": "Entfernungsmesser", "Prinzip": "Infrarotsensor", "Herstel-
ler": "Sharp", "Modell": None, "Maximaldistanz": 0.150}
>>> irs
{'Typ': 'Entfernungsmesser', 'Prinzip': 'Infrarotsensor', 'Hersteller':
'Sharp', 'Modell': None, 'Maximaldistanz': 0.15}
>>>
>>> irs[ "Maximaldistanz"]
0.15
>>> irs[ "Maximale Distanz"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Maximale Distanz'
>>>
```

Der Schlüssel "Maximaldistanz" existiert nicht in diesem Dictionary, weshalb eine `KeyError`-Exception ausgelöst wird.

Andere Namen sind auch *Assoziatives Array*, *Hash* oder *Map*.

6.1.5.2 set – Mengen

Sets sind Mengen wie zum Beispiel die Keys der [Dictionaries](#).

Was sie können, erzählen sie am besten selbst:

```
>>> dir ( set())
['_and_', '__class__', '__class_getitem__', '__contains__', '__delattr__',
'_dir_', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__',
'_getstate__', '__gt__', '__hash__', '__iand__', '__init__', '__init_sub-
class__', '__ior__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__',
'_lt_', '__ne__', '__new__', '__or__', '__rand__', '__reduce__', '__redu-
ce_ex__', '__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__', '__si-
zeof__', '__str__', '__sub__', '__subclasshook__', '__xor__', 'add', 'clear',
'copy', 'difference', 'difference_update', 'discard', 'intersection', 'inter-
section_update', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove',
'symmetric_difference', 'symmetric_difference_update', 'union', 'update']
>>>
```

Sie Namen, die mit einem `_` beginnen und enden, bezeichnen sog. *Magic Methods*. Wir kommen dazu, wenn wir in die faszinierende Welt der objektorientierten Programmierung eintauchen.

6.1.6 Mutable vs. immutable

2DO

6.1.7 Übungen zu Variablen und Datentypen

6.1.7.1 Wie oft wird die Funktion *print* ausgeführt?

```
1 t = 0
2 for i in range(3):
3     t = t+1
4     for j in range(t):
5         print(t)
```

6.1.7.2 Wie lautet der String, der schlussendlich ausgegeben wird?

```
1 t = "Die Sonne scheint heute besonders schön"
2 t = t.split(' ')
3 print (t[3] + ' ' + t[1] + ' ' + t[2])
```

6.1.7.3 Schreiben Sie ein Skript, das ausgibt, welchen Typs die Variable A, B und C sind!

```
1 A = ["Haus", "Auto", "Flugzeug"]
2 B = 4563.0
3 C = "34"
```

6.1.7.4 Was gibt die folgende Programmsequenz aus?

```
1 t = 0.0
2 v = 5.0
3
4 while t < 5:
5     print(t)
6     t += 2.5
```

6.1.7.5 Und was gibt die folgende Programmsequenz aus?

```
1 k = "Der arme Jaeger geht alleine durch den dunklen Wald"
2 t = 0
3 for i in k:
4     if i == 'a':
5         t += 1
6
7 print(t)
```

6.1.7.6 Schreiben Sie ein Programm, das alle Ganzzahlen zwischen 1 und 10 inklusive aufaddiert und das Ergebnis ausgibt!

6.1.7.7 Warum funktioniert folgende Programmsequenz nicht?

```
1 value = 0
2 for elem in ( 1, "zwei", "drei"):
3     value += elem
```

Welche Exception wird wohl "geworfen"?

6.1.7.8 Schreiben Sie ein Programm, das alle geraden Zahlen der Liste und auf jeden Fall die beiden letzten Zahlen ausgibt!

```
L = list( range( 1000))
```

6.1.7.9 Schaltjahrbestimmung: Schreiben Sie ein Programm, das vom User eine Jahreszahl entgegennimmt und dann über eine Funktion auf dem Bildschirm ausgibt, ob es sich um ein Schaltjahr handelt oder nicht!

Hinweis zur Schaltjahrbestimmung:

Jahreszahl ist

- nicht durch 4 teilbar: Kein Schaltjahr
- durch 4 teilbar: Schaltjahr
- durch 100 teilbar: Kein Schaltjahr
- durch 400 teilbar: Schaltjahr

6.1.7.10 Harmonische Reihe: Schreiben Sie eine Funktion, die die Harmonische Reihe

$\sum_{k=1}^n \frac{1}{k}$ berechnet!

6.2 Programmstruktur

6.2.1 Lineare Programmstruktur

Wie der Name schon sagt, handelt es sich bei Programmen dieser Struktur um eine Abfolge von Statements. Man könnte sich ein Skript vorstellen, das auf einem Massenspeicher alle .bak-Files findet und löscht. Oder ein Skript, das vom User Eingaben erwartet, diese verarbeitet und das vllt mehrmals.

Bei letzterem Fall ist es schon weniger wahrscheinlich, dass das Programm unstrukturiert linear ist, muss doch unter Umständen auf Eingaben des Users unterschiedlich reagiert werden. Einfache if-else-Blöcke sind ziemlich wahrscheinlich. Mehr noch, wenn man dem User Dienste zur Verfügung stellt, die er interaktiv nutzen kann, wird man ein GUI oder ein textbasiertes () programmieren:

```
Massenspeicher auswählen.....W
Belegung des Massenspeichers anzeigen.....B
Anzahl Files auf Massenspeicher eruieren...A
Alle BAK-Files löschen.....L

Hilfe.....H

Programm verlassen.....X
-----
Ihre Wahl >
```

Ein solches Programm wird in einer Endlosschleife auf die Eingaben des Users warten und dann die mit einer Wahl verbundenen Operationen durchführen.

6.2.2 Funktionen

6.2.2.1 Motivation und Definition

Die Motivation von Funktionen ist eine einfache: Zusammenfassung von Funktionalität zur komfortablen Wiederverwendung.

6.2.2.2 Formal- und Aktualparameter

Funktion können Argumente haben - sogenannte Formalparameter. Bei Ausführung einer solchen Funktion müssen die entsprechenden Parameter angegeben werden, die jetzt heißen. Eine Funktion kann, muss aber nicht ein Ergebnis liefern. Liefert sie kein Ergebnis, ist sie nur sinnvoll, wenn sie sogenannte Seiteneffekte hat.

Hat eine Funktion keine Formalparameter, "lebt" sie von ihren "Seiteneffekten", d.h. sie tut irgendetwas, ohne dafür zu benötigen. Sie kann, muss aber auch hier wieder kein Ergebnis liefern.

```

1 def sin( x: float):
2     # ALGORITHM FINDING THE SINE OF AN ANGLE GOES HERE
3     return # Sinnlos
4
5 def sin( x: float):
6     # ALGORITHM FINDING THE SINE OF AN ANGLE AND ASSIGNING IT TO result GOES HERE
7     return result # Ok
8
9 def sin( x: float) -> float:
10    # ALGORITHM FINDING THE SINE OF AN ANGLE AND ASSIGNING IT TO result GOES HERE
11    return result # Ok
12
13 def start_controller():
14     # ...
15     return
16
17 def start_controller( initial_value: float):
18     # ...
19     return
20
21 def start_controller() -> bool:
22     # ...
23     return success
24
25 def start_controller( initial_value: float) -> bool:
26     # ...
27     return success

```

Was genau ist bei der folgenden Funktion der Seiteneffekt?

```

1 from random import gauss
2
3 def random_value():
4     value = gauss( 0, 0.1)
5     print( f"{random_value.__qualname__}() returns {value}.")
6     return value

```

Soweit alles, wie es bei anderen Programmiersprachen auch vorkommt, wenn's um Funktionen geht. *Python* kann aber auch hier mehr. Im Folgenden ein paar Beispiele samt ihren Anwendungen.

```
>>> def fun( *, distance, speed):
...     return 42
...
>>> fun( 42, 42)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: fun() takes 0 positional arguments but 2 were given
>>>
>>> def fun( *, distance, speed):
...     return 42
...
>>> fun( distance=42, speed=42)
42
>>>
```

Starten Sie Ihre und versuchen Sie folgende Code-Sequenzen zu programmieren und auszuführen. Versuchen Sie dabei, mit dem Debugger der herauszufinden, was vor sich geht:

```
1 def assign_new_kinematic_data( distance, speed=1.0, accel=1.0):
2     return True
3
4 assign_new_kinematic_data( 100, accel=2.0)
5 assign_new_kinematic_data( distance=100, accel=2.0)
6
7
8 def assign_new_kinematic_data( *, distance, speed=1.0, accel=1.0):
9     return True
10
11 assign_new_kinematic_data( distance=100, accel=2.0)
12
13
14 from math import *
15 def move_to_pose( *args, **kwargs):
16     x, y, z, a, b, c = args
17     if "format" in kwargs and kwargs[ "format"]:
18         a = radians( a)
19         b = radians( b)
20         c = radians( c)
21
22     if "print" in kwargs and kwargs[ "print"] == True:
23         print( f"Position = {(x, y, z)}, Orientation = {a, b, c}.")
24
25     return
26
27 move_to_pose( 1, 2, 3, 0, 0, radians( 90))
28 move_to_pose( 1, 2, 3, 0, 0, 90, format="deg")
29 move_to_pose( 1, 2, 3, 0, 0, 90, format="deg", print=True)
```

6.2.2.3 Return-Werte

Returnwerte können Ergebnisse von Berechnungen sein, aber auch Werte, die die Qualität eines Seiteneffektes beschreiben:

```
1 from math import *
2
3 value = sin( pi)
4 success = start_falcon( 9)
```

6.2.3 Verzweigungen mit *if*

```
1 condition = some_function_returning_a_boolean()
```

```

2 if condition:
3     print( "The function 'some_function_returning_a_boolean()' returned True.")
4
5 else:
6     print( "The function 'some_function_returning_a_boolean()' returned False.")

```

6.2.4 Wiederholungsstrukturen

6.2.4.1 Schleifen mit *while*

```

1 while some_function_returning_a_boolean():
2     print( "The function 'some_function_returning_a_boolean()' still returns True.")
3
4 print( "While-loop has finished.")

```

Um eine Schleife zu beenden, kann auch das `break`-Statement verwendet werden:

```

1 while True:
2     if some_function_returning_a_boolean():
3         print( "The function 'some_function_returning_a_boolean()' has returned True.")
4         break
5
6     print( "The function 'some_function_returning_a_boolean()' still returns False.")
7
8 print( "While-loop has finished.")

```

Schleifen-Teile können per `continue` auch übersprungen werden:

```

1 while True:
2     if some_function_returning_a_boolean():
3         print( "The function 'some_function_returning_a_boolean()' has returned True, skip rest
of the loop.")
4         continue
5
6     print( "The function 'some_function_returning_a_boolean()' still returns False.")

```

Etwas Exotisches ist das [while/else bzw. for/else](#). Hier wird der abschließende `else`-Zweig ausgeführt, wenn die Schleife ordnungsgemäß(!) beendet worden ist - gezeigt an einer `for`-Schleife:

```

1 sentinel = some_magic_number_or_instance()
2 for entry in some_list:
3     if entry == sentinel:
4         break
5
6     process( entry)
7
8 else:
9     raise ValueError( "List is missing terminal flag.")

```

6.2.4.2 Schleifen mit *for*

Im Gegensatz zu `while`-Schleifen, wird hier über sequenzielle Daten iteriert, wie im letzten Kapitel bereits abgedeutet. Wir wollen hier statt über eine Liste über einen String iterieren:

```

1 for c in "Elektrotechnik Dual":
2     print( c)
3 for i, c in enumerate( "Elektrotechnik Dual"):
4     print( f"Character {i} == {c}.")

```

Und - auch `for`-Schleifen kennen die `else`-Klausel.

6.2.5 Callbacks

Callbacks könnte man als indirekte Funktions- oder Methodenaufrufe beschreiben. Ein Beispiel zeigt vllt am besten, was hierbei gemeint ist. Eine praktische Anwendung wehen wir gleich in der Übung [Temperaturrechner - Übung zu 'Programmstruktur'](#).

Wenn wir eine Funktion definieren, die eine beliebige Funktion ausführen soll, dann könnte das so aussehen:

```
5 def caller( callee: callable):
6     print( "Führe die Funktion %s aus." % callee.__name__)
7     callee()
8     return
```

Definieren wir die Funktion

```
9 def callee():
10     print( "Ausführung von %s." % callee.__qualname__)
```

als die Funktion, die von `caller` auszuführen ist, dann erhalten wir durch die Ausführung von

```
caller( callee)
```

die Ausgabe

```
Führe die Funktion callee aus.
Ausführung von callee.
```

6.2.6 Übungen zu Programmstruktur

6.2.6.1 Temperaturrechner

Schreiben Sie ein Programm, das ein Menü ausgibt zur Umrechnung von Temperaturen:

```
1 Celsius nach Kelvin.....1
2 Celsius nach Fahrenheit...2
3 Kelvin nach Celsius.....3
4 Kelvin nach Fahrenheit...4
5 Fahrenheit nach Celsius...5
6 Fahrenheit nach Kelvin...6
7 Programmende.....x
8 -----
9 Ihre Wahl >
```

Das Programm darf nicht "crashen", egal, was der User eingibt!

Hinweis:

$$C = \frac{5}{9}(F - 32)$$

$$C = K - 273.15$$

6.3 Exceptions

Exceptions dienen dazu, auf Unvorhergesehenes "irgendwie" zu reagieren.

```
1 try:
2     x = float( input( "Bitte Zahlenwert für Nenner einer Division eingeben >"))
3     y = 1/x
4     print( f"1/{x} ergibt {y}.")
5
6 except ValueError as err:
```

```

7     print( "Sorry, aber Sie müssen einen Zahlenwert eingeben, nicht '%s'!" % x)
8     # Hier könnten wir die Exception weiterreichen per raise err
9
10 except ZeroDivision as err:
11     print( f"Sorry, kann mit dem Zahlenwert {x} als Nenner einer Division nichts anfangen!")
12     # Hier könnten wir die Exception weiterreichen per raise err

```

6.4 Files

Daten auf Massenspeichern sind in Directorys und Files organisiert. Sie ermöglichen es uns, Daten persistent zu halten, sodass sie beim Herunterfahren des Computers nicht verloren gehen.

Aufgaben zu Files finden Sie weiter unten.

Hier ein paar Fingerübungen, um ein Gefühl für das handling von Files zu bekommen.

Wir erzeugen uns zuerst ein File:

```

1 import os
2
3 f = open( "irgendwas.txt", "w")
4 f.write( "Lösch mich!")
5 f.close()
6
7 print( os.listdir( os.getcwd()))
8                                     # Gibt eine Liste von Files aus, die
9                                     # unser "irgendwas.txt" auch enthalten
10                                    # sollte.

```

Nun lassen wir uns den Inhalt des Files anzeigen, das wir gerade erzeugt haben. Dazu müssen wir es zum Lesen öffnen (Mode "r"):

```

11 with open( "irgendwas.txt", "r") as f:
12     inhalt = f.read()
13     print( inhalt)

```

Ausgabe:

```
Lösch mich
```

Funktioniert!

Wenn wir uns mit den Varianten zum Lesen von Files beschäftigen wollen, müssen wir mehr Inhalt haben. Und wenn wir ein File zeilenweise lesen wollen, müssen wir es auch zeilenweise anlegen. Das tun wir also: Wir schreiben mehrere Zeilen in ein File und lesen das File dann aus:

```

1 with open( "viele_zeilen.txt", "w") as f:
2     for i in range( 42):
3         f.write( "Zeile %02d\n" % (i+1))
4
5 with open( "viele_zeilen.txt", "r") as f:
6     for line in f:
7         line = line.strip()
8         print( line)

```

Ausgabe:

```
Zeile 01
Zeile 02
Zeile 03
Zeile 04
Zeile 05
Zeile 06
Zeile 07
Zeile 08
Zeile 09
Zeile 10
Zeile 11
Zeile 12
Zeile 13
Zeile 14
Zeile 15
Zeile 16
Zeile 17
Zeile 18
Zeile 19
Zeile 20
Zeile 21
Zeile 22
Zeile 23
Zeile 24
Zeile 25
Zeile 26
Zeile 27
Zeile 28
Zeile 29
Zeile 30
Zeile 31
Zeile 32
Zeile 33
Zeile 34
Zeile 35
Zeile 36
Zeile 37
Zeile 38
Zeile 39
Zeile 40
Zeile 41
Zeile 42
```

6.4.1 Übungen zu *Files*

6.4.1.1 File-Lister

Schreiben Sie unter Verwendung einer IDE Ihrer Wahl ein Skript, das die Namen aller Files eines Directories, das als Parameter bei sonstiger Fehlermeldung anzugeben ist, ausgibt.

Wenn sich darunter Files mit der Endung `.txt` finden, wird deren Inhalt auf den Bildschirm ausgegeben. Existieren in dem angegebenen Folder keine solchen Files, wird eine entsprechende Info ausgegeben.

Hinweis:

- Schauen Sie sich das Modul [os](#) und [os.path](#) der [Standard-Library](#) an!
- Damit das wirklich funktioniert, müssen wir lokal arbeiten. Eine gute Gelegenheit, erste Schritte mit VSCode zu machen!

Wann wir dieses Kapitel
gelesen?
→ Roadmap 2024-
WS-25

Grundlagen der objektorientierten Programmierung (OOP)

7.1 Einführung

Objektorientierte Programmierung erfordert eine etwas andere Denkweise: *It's about services, not data*. Man denkt in Abstraktionen von (nicht notwendigerweise) realen Dingen, was sie können, was sie leisten, wenn der User ihre Services nützen möchte. Man denkt weniger in Daten dieser Abstraktionen, wenngleich die einschlägige Literatur zu dieser Denkweise verleiten kann. Hier wird oft vorgeschlagen, sich beim Entwurf einer ein Objekt beschreibender Klasse zu fragen, was dieses Objekt hat ("Has-a"-Relation). Wichtiger ist jedoch das Interface, das beschreibt, was es kann. So kann auch die bei größeren Software-Systemen erforderliche Datenkapselung erreicht werden.

Ein Beispiel: Nehmen Sie an, Sie hätten ein Objekt, das mit einem Namen bezeichnet werden kann. Es hat also einen Namen. Sie werden daher eine Methode definieren, die den Namen liefert, um ihn beispielsweise auszudrucken. Alles bestens hier, das wird funktionieren. Bei größeren Programmen könnten Sie sich aber auch überlegen, eine Methode zu definieren, die den Namen ausdrückt, wofür Sie der entsprechenden Methode ein Drucker-Objekt übergeben. Sie denken also in Services. Dieses Objekt *kann* dann seinen Namen drucken.

Auf diese Weise verbergen Sie die Repräsentation der Daten eines Objekts. Denn die Kenntnis der Repräsentation von Daten, die Kenntnis von Implementierungsdetails erschwert die Wartung größerer Programme.

Wenn Sie dem Mantra [Make it work - make it right - make it fast](#) folgen, wäre das Lesen und anschließende Drucken des Namens *Make it work* und das Umschreiben in eine Fähigkeit des Objekts das *Make it right*. Oft reicht aber auch das *Make it work* und im [Zen of Python](#) heißt es ja auch:

Now is better than never.
Although never is often better than **right** now.

Dafür gibt's auch ein Akronym: [YAGNI](#).

7.2 Ein einführendes Beispiel

Wir hatten andernorts eine Funktion geschrieben, die die y-Werte einer Geraden berechnet. Dabei mussten die Parameter *k* und *d* immer mitangegeben werden. Das ist nicht nur lästig, sondern auch wenige intuitiv, sollen sich diese Parameter doch gar nicht ändern.

Was, wenn wir ein Objekt anlegen könnten; das

- mit *k* und *d* konfiguriert wird und das
- eine Funktion besitzt, die nur den x-Wert entgegennimmt, um den entsprechenden y-Wert zu berechnen?

Wir würden ein solches Objekt so erzeugen und verwenden wollen:

```
1 g = Gerade( k=0.5, d=1)
2                               g ist eine Instanz der Klasse Gerade
3 for x in range( -10, 10):
```

```
4 print( f"{x} → {g.y( x)}")
```

Cool, nicht?

Aber wie definiert man sowas?

7.3 Kriterien objektorientierter Sprachen

Die Techniken

- Kapselung im Sinne von Information Hiding
- **Klassen und Objekte**
- Polymorphie

unterstützen die Prinzipien

- Abstraktion
- Modularität
- Hierarchie

und damit das Erreichen der Ziele

- Zuverlässigkeit
- [Wiederverwendbarkeit](#)
- Erweiterbarkeit (s. [Das Open-Closed-Prinzip](#)¹²)

7.4 Klassen

Eine Klasse *Gerade* würde so definiert werden können:

```
1 class Gerade:
2     def __init__( self, k, d):          # Konstruktor oder kurz Ctor
3         self.__k = k                  # Attribut
4         self.__d = d                  # Attribut
5         return
6
7     def y( self, x):                   # Methode. def definiert Funktionen u. Methoden
8         return self.__k * x + self.__d
```

Sie stellen sich sicher die Frage, was es mit diesem `self` auf sich hat. Nun das ist der „Pointer“ auf die Instanz selbst, die Daten. Wir können ja viele Geraden haben, die aber alle die gleichen Methoden haben.

Die folgende Fig. 3 einer Debugging-Session mag das illustrieren:

12 Es lohnt sich, in [Prinzipien der Softwaretechnik](#) zu stöbern! Der Autor wahrt die abgeklärte Distanz zu "Modernem" und beurteilt die Themen, die er bespricht, nach ihrem Nutzen. Er nimmt auf Bezug auf Bertrand Meyer, den Schöpfer der Programmiersprache *Eiffel*. Meyer ist ein Fixstern am Himmel der Informatikgrößen.

7 Grundlagen der objektorientierten Programmierung (OOP)

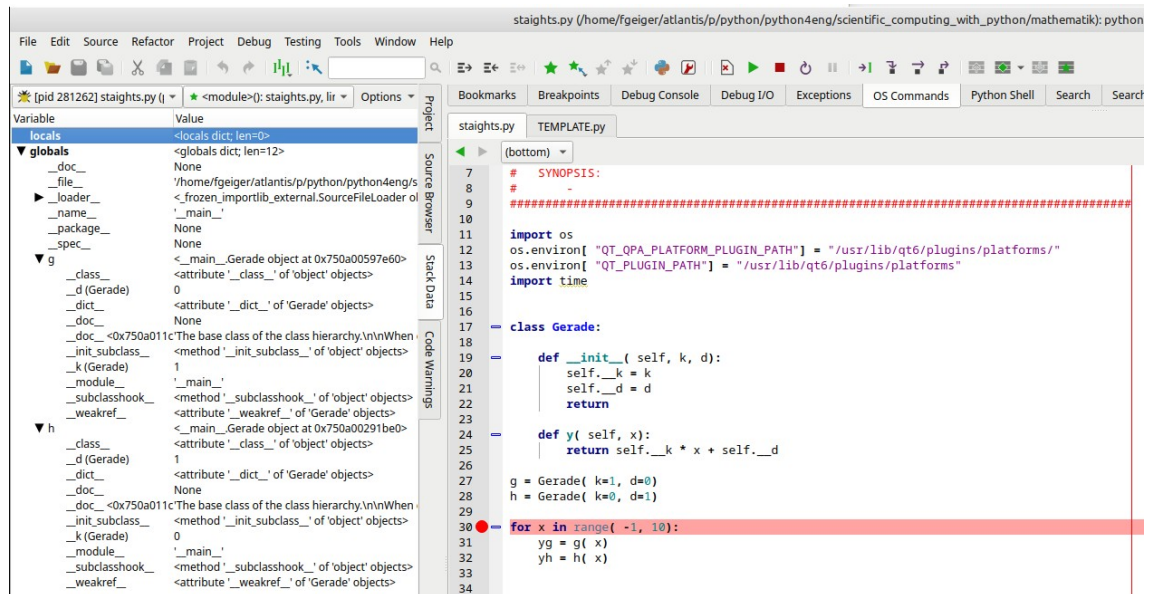


Fig. 3

8 Grafische Ausgabe mit *matplotlib*

8.1 Dokumentation

Die Dokumentation zu matplotlib findet sich unter <https://matplotlib.org/stable/index.html>. Hier finden Sie:

- User Guide
- Tutorials
- Beispiele
- Referenz

Unter <https://matplotlib.org/stable/gallery/index.html> wird man vermutlich am schnellsten zum Ziel kommen, wenn man ein bestimmtes Problem hat.

8.2 Rezepte

8.2.1 Balkendiagramme plotten

```

1 import os
2 os.environ[ "QT_QPA_PLATFORM_PLUGIN_PATH" ] = "/usr/lib/qt6/plugins/platforms/"
3 os.environ[ "QT_PLUGIN_PATH" ] = "/usr/lib/qt6/plugins/platforms"
4
5
6 import matplotlib.pyplot as plt
7
8
9 #.....
10 #   Wir definieren unser Lager
11 #
12 lagerbestand = (("Äpfel", 50, "green"), ("Bananen", 99, "yellow"), ("Birnen", 2, "gold"),
13                ("Kirschen", 52, "red"), ("Orangen", 0, "orange"))
14 #.....
15 #   Wir holen uns die Figure, die die Axis enthält, in die wir plotten
16 #
17 figure, axis = plt.subplots()
18
19 #.....
20 #   Unser Lager ist sehr übersichtlich definiert. plot erwartet aber einzelne Listen:
21 #   Eine für die Namen, eine für die Mengen, eine für die Farben usw. Also müssen wir
22 #   unser Lager aufbereiten.
23 #
24 frucht_namen = [ item[ 0 ] for item in lagerbestand ]
25 frucht_mengen = [ item[ 1 ] for item in lagerbestand ]
26 frucht_farben = [ item[ 2 ] for item in lagerbestand ]
27
28 #.....
29 #   Wir plotten endlich, eine grafische Ausgabe erfolgt allerdings noch nicht!
30 #
31 axis.bar( frucht_namen, frucht_mengen, label=frucht_namen, color=frucht_farben )
32 axis.set_title( "Lagerbestand Obst" )
33 axis.set_ylabel( "Anzahl Steigen" )
34 axis.legend( title="Fruchtfarben" )
35
36 #.....
37 #   Jetzt noch die Mengenangaben über jeden Balken
38 #
39 for i in range( len( frucht_namen ) ):
```

8 Grafische Ausgabe mit matplotlib

```
40     plt.text( i, frucht_mengen[ i] + 1, frucht_mengen[ i])
41
42 #.....
43 #    Und zack - Ausgabe
44 #
45 plt.show()
```

9 Librarys fürs wissenschaftliche Rechnen

9.1 NumPy

9.1.1 Ein Beispiel zur Motivation

9.1.1.1 Grafisches Ausgabe der Funktion $y = \sin^2(x)$

Das Intervall, in dem die Funktion gezeichnet werden soll, ist

$$-2\pi \leq x \leq 2\pi$$

Für die grafische Ausgabe verwenden wir *matplotlib* (s. Kap. 8).

9.1.1.1.1 Variante 1

Wir brauchen ein Δx , das davon abhängt, wie viele Punkte n wir in diesem Intervall zeichnen möchten:

$$\Delta x = \frac{x_{\max} - x_{\min}}{n - 1}$$

Damit können wir schon programmieren:

```
1 import os
2 import time
3
4
5 import math
6 import matplotlib.pyplot as plt
7
8
9 # Variant 1 - Python
10 #
11 x_min, x_max = -2. * math.pi, 2. * math.pi
12 n = 1000
13 xx = [ 0.] * n
14 yy = [ 0.] * n
15 dx = (x_max - x_min)/(n-1)
16
17 dt = time.time()
18 for i in range( n):
19     xx[i] = x_min + i * dx
20     yy[i] = math.sin( xx[i])**2
21
22 plt.plot( xx, yy)
23 dt = time.time() - dt
24 plt.show()
25 print( f"It took me {dt*1000:.3f} ms to cal and plot variant 1.")
```

9.1.1.1.2 Variante 1 + 2

```
1 import os
2 import time
3
4
5 import math
6 import matplotlib.pyplot as plt
7
8
```

```

9 # Variant 1 - Python
10 #
11 x_min, x_max = -2. * math.pi, 2. * math.pi
12 n = 1000
13 xx = [ 0.] * n
14 yy = [ 0.] * n
15 dx = (x_max - x_min)/(n-1)
16
17 dt = time.time()
18 for i in range( n):
19     xx[i] = x_min + i * dx
20     yy[i] = math.sin( xx[i])**2
21
22 plt.plot( xx, yy)
23 dt = time.time() - dt
24 plt.show()
25 print( f"It took me {dt*1000:.3f} ms to cal and plot variant 1.")
26
27 # Variant 2 - NumPy
28 #
29 import numpy as np
30 x_min, x_max = -2. * math.pi, 2. * math.pi
31 x = np.linspace( x_min, x_max, n)
32 dt = time.time()
33 y = np.sin( x)**2
34 plt.plot( x, y)
35 dt = time.time() - dt
36 plt.show()
37 print( f"It took me {dt*1000:.3f} ms to cal and plot variant 2.")

```

Welche Laufzeiten messen Sie? Die NumPy-Variante sollte um eine Größenordnung schneller sein.

9.1.2 Aufgabe sinc-Funktion

Stellen Sie Funktion

$$y = \frac{\sin x}{x}$$

im Intervall [-20, 20] grafisch dar!

9.1.3 Matrizen – erste Schritte

Gegeben sei die Matrix

$$\underline{A} = \begin{bmatrix} 0 & 2 & -1 \\ 2 & 3 & 1 \\ 6 & 42 & 8 \end{bmatrix}$$

1. Programmieren Sie diese Matrix und finden Sie programmatisch Folgendes heraus:
 1. Welche Dimension hat die Matrix?
 2. Wie lautet die 3. Zeile?
 3. Wie lautet die 2. Spalte?
2. Multiplizieren Sie mit ihrer Inversen und stellen Sie sicher, dass auch wirklich die Einheitsmatrix dabei rauskommt.

Hinweis

Ihr Programm müsste die folgende Ausgabe verursachen:

```
[[ 0  2 -1]
 [ 2  3  1]
 [ 6 42  8]]
A is of dim (3, 3)
Last row of A is [ 6 42  8]
2nd column of A is [ 2  3 42]
The inverse of A is [[ 0.20930233  0.6744186 -0.05813953]
 [ 0.11627907 -0.06976744  0.02325581]
 [-0.76744186 -0.13953488  0.04651163]]
```

Dazu müsste es in etwa so aussehen:

```
1 import numpy as np
2
3
4 A = np.array( [[0, 2, -1], [2, 3, 1], [6, 42, 8]])
5 print( A)
6
7 print( f"A is of dim {A.shape}")
8
9 print( f"Last row of A is {A[ -1, :]}")
10 print( f"2nd column of A is {A[ :, 1]}")
11
12 iA = np.linalg.inv( A)
13 print( f"The inverse of A is {iA}")
14
15 assert np.allclose( A @ iA, np.eye( 3))
```

9.2 SymPy

SymPy ist ein Paket zum symbolischen Rechnen. Die [Doku](#) ist ehr gut und bietet auch eine Sektion [Tutorials](#).

10 Python als Taschenrechner

10.1 Mengen 1

Die folgenden Aufgaben stammen größtenteils aus dem [Skript zum Mathematischen Praktikum](#).

Gegeben sind die Mengen $A = \{3, 4, 5, 6, 7\}$ und $B = \{1, 6, 7, 8, 9\}$. Schreiben Sie ein Python-Skript, das

- die Vereinigungsmenge
- die Schnittmenge
- die Differenzmenge $A \setminus B$ berechnet.

10.2 Mengen 2

Geben Sie die folgenden Mengen in aufzählender Darstellung an, indem Sie sie ein Python-Skript berechnen lassen:

- $\{2n \mid n \in \mathbb{N}, 0 \leq n \leq 4\}$
- $\{2^{1/n} \mid n \in \mathbb{N}, 0 \leq n \leq 4\}$
- $\{x \in \mathbb{N} \mid 1 \leq x \leq 7\}$
- $\{x \in \mathbb{N} \mid 2 \leq x \leq 5\} \cup \{x \in \mathbb{N} \mid 4 \leq x \leq 7\}$
- $\{1, 2\} \times \{5, 6\}$
- $\{x \in \mathbb{R} \mid 2x^2 + 3x = 2\}$
- $\{x \in \mathbb{N} \mid |x| \leq 4\}$

10.3 Funktionen

10.3.1 Geradengleichung – Variante einfache Funktion

Schreiben Sie eine Funktion, die es erlaubt, die Y-Werte einer Geradengleichung der Form

$$y = kx + d$$

auszugeben, wenn die entsprechenden x-Werte angegeben werden.

10.3.2 Geradengleichung – Variante objektorientiert

Um dieses Beispiel lösen zu können, arbeiten Sie bitte zuerst Kap. 7 durch.

Schreiben Sie eine Klasse, die es erlaubt, die Y-Werte einer Geradengleichung der Form

$$y = kx + d$$

auszugeben, wenn die entsprechenden x-Werte angegeben werden. Die Parameter k und d sollen dem Ctor übergeben werden können und sind unveränderlich.

10.3.3 Quadratische Gleichung

Schreiben Sie eine Funktion, die aus den Koeffizienten einer quadratischen Gleichung die beiden Lösungen berechnet und überlegen Sie sich dabei, wie Sie mit negativen Diskriminanten umgehen wollen. Führen Sie die Funktion in einem Hauptprogramm aus.

1. Brechen Sie in einer ersten Version "mit Würde" (= ohne Traceback) ab!
2. In einer zweiten Version liefern Sie die beiden konjugiert-komplexen Lösungen!

10.3.4 Quadratische Gleichung mit cmath

Schreiben Sie eine Funktion, die die Koeffizienten `a`, `b` und `c` einer quadratischen Gleichung entgegennimmt und die Lösungen liefert. Konjugiert-komplexe Lösungen müssen ebenso möglich sein!

10.3.5 Dreiecksfläche

Schreiben Sie eine Funktion `dreiecksflaeche` (und führen Sie sie dann aus), die die 3 Eckpunkte

$$P_1=(x_1, y_1), P_2=(x_2, y_2), P_3=(x_3, y_3)$$

entgegennimmt und per

$$A=\sqrt{s(s-a)(s-b)(s-c)}$$

mit

$$s=\frac{a+b+c}{2}$$

berechnet.

Hinweise:

- `a`, `b`, `c` sind die Seitenlängen des Dreiecks.
- Da hier mehrere Schritte notwendig sind, um an das Ergebnis zu kommen, sollten Sie weitere Funktion überlegen, die Sie dann in `dreiecksflaeche` verwenden.

10.3.6 Volumina

Schreiben Sie eine App, die aus einem File Art und Abmessungen geometrischer Körper liest und deren Volumina berechnet!

Erstellen Sie hierzu ein File mit ein paar Einträgen und ermöglichen Sie dem User, dass er über einen Button oder Menüpunkt einen Text-Editor starten kann, mit dem er genau dieses File editieren kann.

10.3.7 Maximum einer Parabel

Berechnen Sie das Maximum der Funktion . Stellen Sie Stammfunktion und 1. Ableitung grafisch dar.

10.4 Gleichungen und Gleichungssysteme

Wir wollen die Nullstellen der Funktion finden, d.h. wir haben die Gleichung zu lösen.

Wir verwenden [SymPy](#) und plotten die Funktion als erstes:

```
1 import sympy
```

10 Python als Taschenrechner

```
2 from sympy.plotting import plot
3
4 x = sympy.Symbol( "x", real=True)
5 f = x**4 - 1
6 pf = plot( f, xlim=(-5, 5), ylim=(-3, 3))
```

Nun lösen wir die Gleichung :

```
1 sols = sympy.solve( f, x)
2 print( sols)
```

Wir sehen also die beiden Lösungen, die wir auch in der Grafik sehen. Die anderen sehen wir in der Grafik nicht. Wir hier dem Taschenrechner glaubt, ist verloren.

Alle anderen wissen, dass diese Gleichungen 4 Lösungen haben müssen.

Wie aber sieht's aus mit einem System von Gleichungen in mehreren Variablen?

Versuchen wir das Gleichungssystem

$$\begin{aligned} x+5y-2 &= 0 \\ -3x+6y-15 &= 0 \end{aligned}$$

zu lösen!

Jetzt Du :-)

```
1 import sympy
2 from sympy.plotting import plot
3
4 x = sympy.Symbol( "x", real=True)
5 sols = sym.solve((x + 5 * y - 2, -3 * x + 6 * y - 15), (x, y))
6 print( sols)
```

10.5 Vektorrechnung

10.5.1 Eigenwerte und Vektoren

Bei diesem Problem geht es darum, Werte und Vektoren zu finden, für die

$$\underline{A} \cdot \underline{x} = \lambda \cdot \underline{x}$$

gilt. Werte , die diese Gleichung erfüllen, heißen Eigenwerte, die zugeordneten Vektoren Eigenvektoren.

Es handelt sich bei Eigenvektoren also um Vektoren, die nur in ihrer Länge verändert werden, wenn sie mit multipliziert werden, aber dadurch nicht gedreht werden.

Mit anderen Worten: Wir sind interessiert an Transformationen $\underline{A} \underline{x}$, die $\underline{x}$ nur in der Länge ändern, nicht aber in seiner Orientierung. Die Eigenvektoren sind dann solche Vektoren und λ sind die Skalierungsfaktoren.

Wir wollen uns das grafisch veranschaulichen:

```
1 import os
2 os.environ[ "QT_QPA_PLATFORM_PLUGIN_PATH" ] = "/usr/lib/qt6/plugins/platforms/"
3 os.environ[ "QT_PLUGIN_PATH" ] = "/usr/lib/qt6/plugins/platforms"
4
```

```

5 import numpy as np
6 import matplotlib.pyplot as plt
7
8
9 #plt.style.use( 'seaborn-poster')
10
11 def plot_vectortransform( A, x, xlim, ylim):
12     b = np.dot( A, x)
13
14     plt.figure( figsize = (10, 6))
15     plt.quiver( 0,0,x[0],x[1],\
16               color='k',angles='xy',\
17               scale_units='xy',scale=1,\
18               label='Original vector'
19     )
20     plt.quiver( 0, 0, b[0], b[1],\
21               color='g',angles='xy',\
22               scale_units='xy',scale=1,\
23               label='Transformed vector'
24     )
25     plt.xlim( xlim)
26     plt.ylim( ylim)
27     plt.xlabel( 'x')
28     plt.ylabel( 'y')
29     plt.legend()
30     plt.show()
31
32
33 A = np.array( [[2, 0], [0, 1]])
34 x = np.array( [[1], [1]])
35 plot_vectortransform( A, x, (0,3), (0,2))
36
37 x = np.array( [[1], [0]])
38 plot_vectortransform( A, x, (0,3), (-0.5,0.5))

```

Die Gleichung

$$\underline{A} \cdot \underline{x} = \lambda \cdot \underline{x}$$

bzw.,

$$(\underline{A} - \lambda \underline{E}) \underline{x} = 0$$

hat nicht-triviale Lösungen für

$$\underline{A} - \lambda \underline{E} = 0$$

Wenn man die Eigenwerte hat, setzt man sie ein in $\underline{A} - \lambda \underline{E} = 0$ ($\underline{x}$ soll ja ungleich 0 sein).

Versuchen Sie die Lösung für

$$\underline{A} = \begin{bmatrix} 0 & 2 \\ 3 & 3 \end{bmatrix}$$

aber von Hand!

Sie müssten auf die **Lösung**

$$\lambda_1 = 4, \lambda_2 = -1$$

und die Eigenvektoren

$$\underline{x}_1 = k_1 \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \underline{x}_2 = k_2 \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

kommen.

Und jetzt lassen wir Python rechnen:

```
1 A = np.array( [[0, 2],[2, 3]])
2 _lambda, v = np.linalg.eig( A)
3 print( f"Eigenvalues:  {_lambda}")
4 print( f"Eigenvectors: {v}")
```

ergibt

```
1 Eigenvalues:  [-1.  4.]
2 Eigenvectors: [[-0.89442719 -0.4472136 ]
3 [ 0.4472136 -0.89442719]]
```

10.6 Dreiphasensystem animiert

Schreiben Sie Script, das das Dreiphasensystem als rotierendes Zeiger-Tripel ausgibt, wie gezeigt in Fig. 4. Erzeugen Sie auch eine MP4-Datei davon, die Sie an Interessierte schicken könnten.

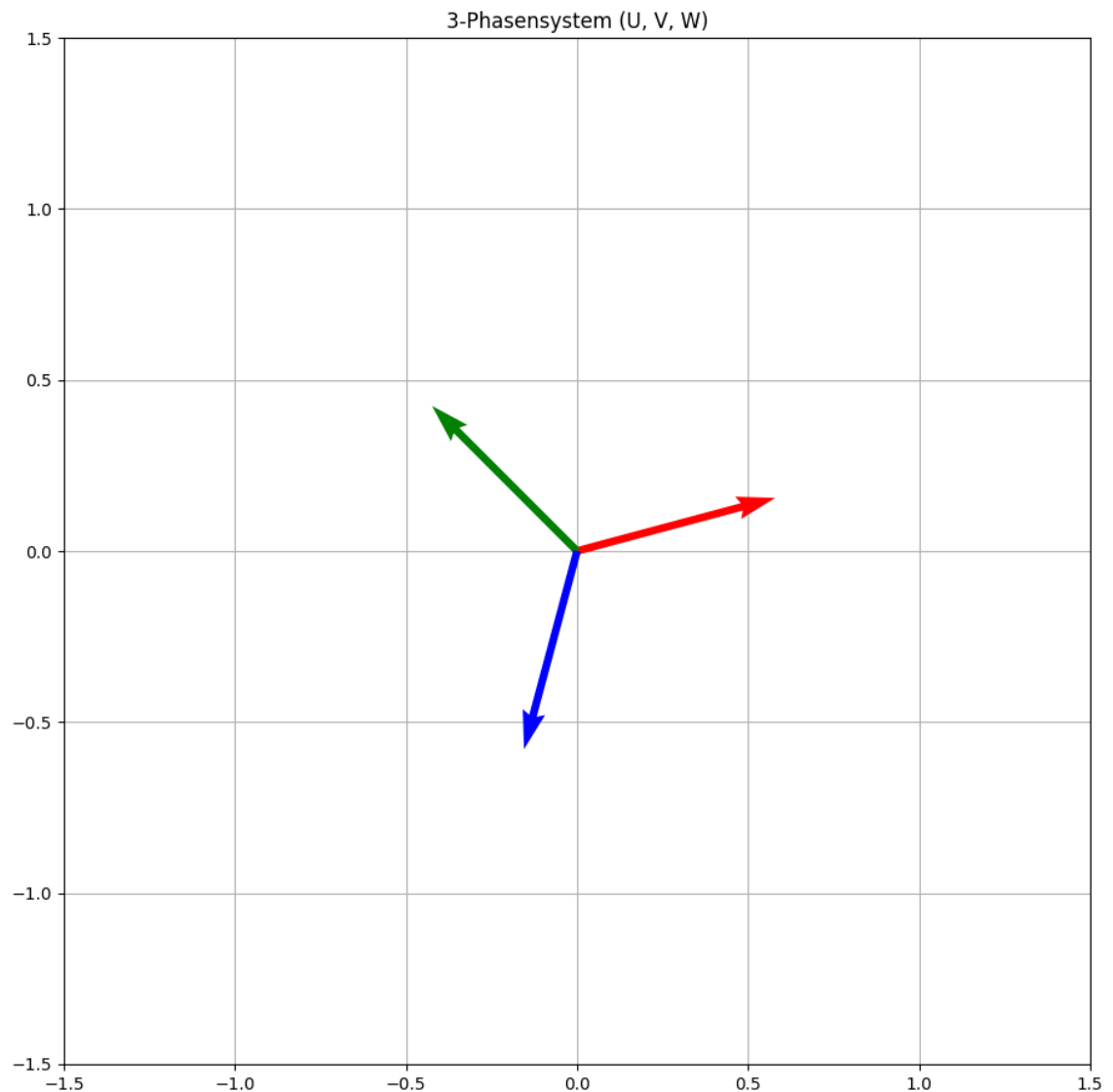


Fig. 4

Versuchen Sie dabei auch eine objektorientierte Variante zu realisieren!

11 Literatur

12 Anhang: Fundstücke

Der Autor von [One Liners Python Edition](#) schreibt:

- I've been immersed in the world of Python programming for approximately three years. During this time, I've come to appreciate the elegance and power of this versatile language. In this post, designed for both fun and education, I'll be presenting a collection of one-liner Python code snippets. Whether you're a seasoned developer or a beginner, these concise lines of code offer insights into the simplicity and effectiveness of Python, demonstrating how a single line can accomplish what might take several lines in other languages.
- [Python in a Nutshell](#) ist endlich in einer Neuauflage erschienen. Die angeführten Autoren sind langjährige "Python-Größen".
- [Python 3.12: Cool New Features for You to Try](#).

13 Anhang: Dokumentenverzeichnis

Index

A	Lineare Programmstruktur.....46
Anhang: Dokumentenverzeichnis.....75	Linker.....1
Anhang: Fundstücke.....73	Literatur.....71
B	M
Balkendiagramme plotten.....59	Matrizen.....62
C	Maximum einer Parabel.....66
Compiler.....1	Mengen 1.....65
D	Mengen 2.....65
Debugger.....1	N
Dreiecksfläche.....66	NumPy.....61
Dreiphasensystem animiert.....70	P
E	Programmieren, was ist das?.....11
Editor.....1	Programmstruktur.....46
Eigenwerte und Vektoren.....67	Python als Taschenrechner.....65
Einführung in Python – Erste Schritte.....17	Q
Exceptions.....50	Quadratische Gleichung.....65
F	Quadratische Gleichung mit cmath.....66
Files.....51	R
Funktionen.....47, 65	Roadmap 2024-WS-25.....9
G	S
Geradengleichung – Variante einfache Funkti- on.....65	sinc-Funktion.....62
Geradengleichung – Variante objektorientiert65	SymPy.....63
Gleichungen und Gleichungssysteme.....66	T
Glossar.....1	Temperaturrechner.....50
Grafische Ausgabe mit matplotlib.....59	U
Grundlagen.....33	Übungen.....50
Grundlagen der objektorientierten Program- mierung (OOP).....55	V
I	Vektorrechnung.....67
IDE.....1	Volumina.....66
L	67
Libraris fürs wissenschaftliche Rechnen.....61	

